1. Traballo de Machine Learning - Sebastián Táboas Pazo e Diego Serantes Mitkova

CAPTION

Neste código hai bastantes celas que tardan un tempo considerable en carregar (entre 30 s e 3 min) e a última sección que denomino `Apéndice Cross Validation en redes neuronais` nun ordenador relativamente potente tarda un promedio de ~7 horas --> **NON COMPILAR APÉNDICE**. Na sección 4 hai bastantes liñas comentadas tamén para reducir o custe computacional o que pode xerar problemas nalgún plot que dea crash (erro tipo variable not defined) ou que simplemente non plotee nada.

1.1 1. Depuración de datos

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

import sys
import os
sys.path.append(os.getcwd())
```

```
# https://www.kaggle.com/datasets/samartalwar/caffeine-crash-and-jitters-energy-drinks-and-anxiety
df = pd.read_csv('caffeine_crash_and_anxiety_telemetry.csv')
df.head()
```

Columns 1–8 of 19

	user_id	age	gender	cyp1a2_metabolizer_type	caffeine_tolerance	baseline_anxiety_score	prior_sleep_hours
0	CAF-00001	27	Female	Fast	High	2.4	7.0
1	CAF-00002	57	Female	Fast	Low	9.1	7.1
2	CAF-00003	39	Male	Normal	Low	2.9	5.1
3	CAF-00004	34	Female	Fast	Moderate	3.8	5.0
4	CAF-00005	22	Female	Slow	High	2.0	6.3

Columns 9–15 of 19 · first column repeated

	daily_hydration_liters	primary_caffeine_source	caffeine_mg	sugar_grams	consumption_duration_min	heart_rate_spike_bpm	jitters_intensity_score
0	2.29	Brewed Drip Coffee	138	8.7	36	2.0	1.0
1	2.41	Energy Drink (Sugar-Free)	164	0.0	3	2.4	2.0
2	2.07	Energy Drink (Full Sugar)	179	36.9	10	3.8	4.8
3	2.34	Pre-Workout Supplement	408	1.6	3	6.8	4.2
4	1.92	Energy Drink (Full Sugar)	170	50.2	11	2.0	2.0

Columns 16–19 of 19 · first column repeated

	acute_anxiety_score	crash_onset_hours	crash_severity_score	crash_risk_level
0	1.0	7.5	4.1	Mild Afternoon Fatigue
1	5.5	7.5	2.5	Sustained / Negligible Crash
2	5.5	3.6	7.9	Moderate Crash & Lethargy
3	7.3	7.5	10.0	Severe Crash & Tachycardia

	acute_anxiety_score	crash_onset_hours	crash_severity_score	crash_risk_level
4	1.1	2.6	10.0	Severe Crash & Tachycardia

```
df.info() # sinala automaticamente onde hai valores nulos e o tipo de columna
```

```
<class 'pandas.DataFrame'>
RangeIndex: 8000 entries, 0 to 7999
Data columns (total 18 columns):
 #   Column                    Non-Null Count  Dtype  
---  ------                    --------------  -----  
 0   user_id                   8000 non-null   str    
 1   age                       8000 non-null   int64  
 2   gender                    8000 non-null   str    
 3   cyp1a2_metabolizer_type   8000 non-null   str    
 4   caffeine_tolerance        8000 non-null   str    
 5   baseline_anxiety_score    8000 non-null   float64
 6   prior_sleep_hours         8000 non-null   float64
 7   daily_hydration_liters    8000 non-null   float64
 8   primary_caffeine_source   8000 non-null   str    
 9   caffeine_mg               8000 non-null   int64  
 10  sugar_grams               8000 non-null   float64
 11  consumption_duration_min  8000 non-null   int64  
 12  heart_rate_spike_bpm      8000 non-null   float64
 13  jitters_intensity_score   8000 non-null   float64
 14  acute_anxiety_score       8000 non-null   float64
 15  crash_onset_hours         8000 non-null   float64
 16  crash_severity_score      8000 non-null   float64
 17  crash_risk_level          8000 non-null   str    
dtypes: float64(9), int64(3), str(6)
memory usage: 1.1 MB
```

```
# agora vemos datos máis pormenorizadamente, e.g., para "crash_risk_level" porque pretenderemos predicila, podemos ver calquera das columnas que nos devolve df.info()
print(f"{'crash_risk_level'}: \n{df['crash_risk_level'].describe()}")
df.crash_risk_level.unique()
```

```
crash_risk_level:
count                         8000
unique                           4
top       Moderate Crash & Lethargy
freq                          2426
Name: crash_risk_level, dtype: object

<StringArray>
[      'Mild Afternoon Fatigue', 'Sustained / Negligible Crash',
    'Moderate Crash & Lethargy',   'Severe Crash & Tachycardia']
Length: 4, dtype: str
```

Vemos que no noso dataset non temos columnas que conteñan celas baleiras. As columnas "user_id", " gender", "cyp1a2_metabolizer_type", "caffeine_tolerance", "primary_caffeine_source" e "crash_risk_level" son strings. Estes datos teremos que depuralos, no caso de "user_id" non nos aportará nada de información e "crash_risk_level" non será necesario tocala en principio porque aplicaremos as nosas técnicas tal cal sobre a variable str.

```
for i in range(1,len(df.columns)):
    if df[df.columns[i]].dtypes == 'str':
        print(f'Na columna "{df.columns[i]}" pódense tomar os seguintes valores
{df[df.columns[i]].unique()}')

s = 0 # isto serve para contar cantos suxeitos satisfan certo valor
for i in range(len(df)):
    if df['crash_risk_level'].iloc[i] == 'Severe Crash & Tachycardia':
        s += 1
print(f'En gender hai {s} Non-Binary') # podo poñer isto máis xeral 2377 , 1036,
2426, 2161
```

```
Na columna <<gender>> pódense tomar os seguintes valores <StringArray>
['Female', 'Male', 'Non-Binary']
Length: 3, dtype: str
Na columna <<cyp1a2_metabolizer_type>> pódense tomar os seguintes valores
<StringArray>
['Fast', 'Normal', 'Slow']
Length: 3, dtype: str
Na columna <<caffeine_tolerance>> pódense tomar os seguintes valores <StringArray>
['High', 'Low', 'Moderate']
Length: 3, dtype: str
Na columna <<primary_caffeine_source>> pódense tomar os seguintes valores
<StringArray>
[         'Brewed Drip Coffee',   'Energy Drink (Sugar-Free)',
   'Energy Drink (Full Sugar)',      'Pre-Workout Supplement',
 'Espresso / Specialty Drinks',          'Matcha / Green Tea']
Length: 6, dtype: str
Na columna <<crash_risk_level>> pódense tomar os seguintes valores <StringArray>
[      'Mild Afternoon Fatigue', 'Sustained / Negligible Crash',
    'Moderate Crash & Lethargy',   'Severe Crash & Tachycardia']
Length: 4, dtype: str
En gender hai 2161 Non-Binary
```

Segundo os valores que vemos, as variables "cyp1a2_metabolizer_type" e "caffeine_tolerance" pódense poñer perfectamente como datos numéricos cun diccionario. Para os casos " gender", "primary_caffeine_source" e "crash_risk_level" o mellor é traballar con variables dummies. Decatémonos que a primeira columna non nos aporta ningunha información, é un código de identificación do suxeito avaliado. O mellor é eliminalo.

```
gender_dummies = pd.get_dummies(df['gender'], prefix='gender', dtype=int)
primary_caffeine_source_dummies = pd.get_dummies(df['primary_caffeine_source'],
prefix='primary_caffeine_source', dtype=int)
crash_risk_level_dummies = pd.get_dummies(df['crash_risk_level'],
prefix='crash_risk_level', dtype=int)
# En principio non será necesario para os calculos utilizar crash_risk_level_dummies,
si será interesante para poñer a matriz de correlación

df['cyp1a2_metabolizer_type'] = df['cyp1a2_metabolizer_type'].map({'Fast':2,
'Normal':1, 'Slow':0})
df['cyp1a2_metabolizer_type'] = df['cyp1a2_metabolizer_type'].fillna(0) # por se
acaso aparecen NaN pois que sexan lexibles numericamente

df['caffeine_tolerance'] = df['caffeine_tolerance'].map({'High':2, 'Moderate':1,
'Low':0})
df['caffeine_tolerance'] = df['caffeine_tolerance'].fillna(0)

df = df.rename(columns = {'cyp1a2_metabolizer_type':
'cyp1a2_metabolizer_speed_type'})

# como na sección seguinte aínda nos vai interesar utilizar o noso dataframe df con
algunhas variables str utilizamos unha copia para cando
# sexa necesario empregar os dummies que será nos puntos 3 e 4. A columna "user_id"
só nos entorpece; eliminámola directamente.
Df = df.copy()
Df = pd.concat([Df.drop(columns=['user_id','gender','primary_caffeine_source']),
                                gender_dummies, primary_caffeine_source_dummies],
                                axis=1)

Df.head()
```

Columns 1–8 of 22

	age	cyp1a2_metabolizer_speed_type	caffeine_tolerance	baseline_anxiety_score	prior_sleep_hours	daily_hydration_liters	caffeine_mg
0	27	2	2	2.4	7.0	2.29	138
1	57	2	0	9.1	7.1	2.41	164
2	39	1	0	2.9	5.1	2.07	179
3	34	2	1	3.8	5.0	2.34	408
4	22	0	2	2.0	6.3	1.92	170

Columns 9–15 of 22 · first column repeated

	sugar_grams	consumption_duration_min	heart_rate_spike_bpm	...	crash_risk_level	gender_Female	gender_Male
0	8.7	36	2.0	...	Mild Afternoon Fatigue	1	0

	sugar_grams	consumption_duration_min	heart_rate_spike_bpm	...	crash_risk_level	gender_Female	gender_Male
1	0.0	3	2.4	...	Sustained / Negligible Crash	1	0
2	36.9	10	3.8	...	Moderate Crash & Lethargy	0	1
3	1.6	3	6.8	...	Severe Crash & Tachycardia	1	0
4	50.2	11	2.0	...	Severe Crash & Tachycardia	1	0

Columns 16–22 of 22 · first column repeated

	gender_Non-Binary	primary_caffeine_source_Brewed Drip Coffee	primary_caffeine_source_Energy Drink (Full Sugar)	primary_caffeine_source_Energy Drink (Sugar-Free)	primary_caffeine_source_Espresso / Specialty Drinks	primary_caffeine_source_Matcha / Green Tea	primary_caffeine_source_Pre-Workout Supplement
0	0	1	0	0	0	0	0
1	0	0	0	1	0	0	0
2	0	0	1	0	0	0	0
3	0	0	0	0	0	0	1
4	0	0	1	0	0	0	0

5 rows × 24 columns

Por último comprobamos que non haxa dexeneración nos nosos datos (filas duplicadas).

```
if df.duplicated().sum() > 0:
    print(f'Hai {df.duplicated().sum()} filas duplicadas (dexeneración de datos) no
DataFrame. Logo temos que depurar estos datos duplicados, e.g. eliminándoos')
else:
    print('Non hai filas duplicadas. Nada que facer.')
```

```
Non hai filas duplicadas. Nada que facer.
```

1.2 2. Representacións estatísticas

```
numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
fig, axs = plt.subplots(3,5, figsize=(30, 14))

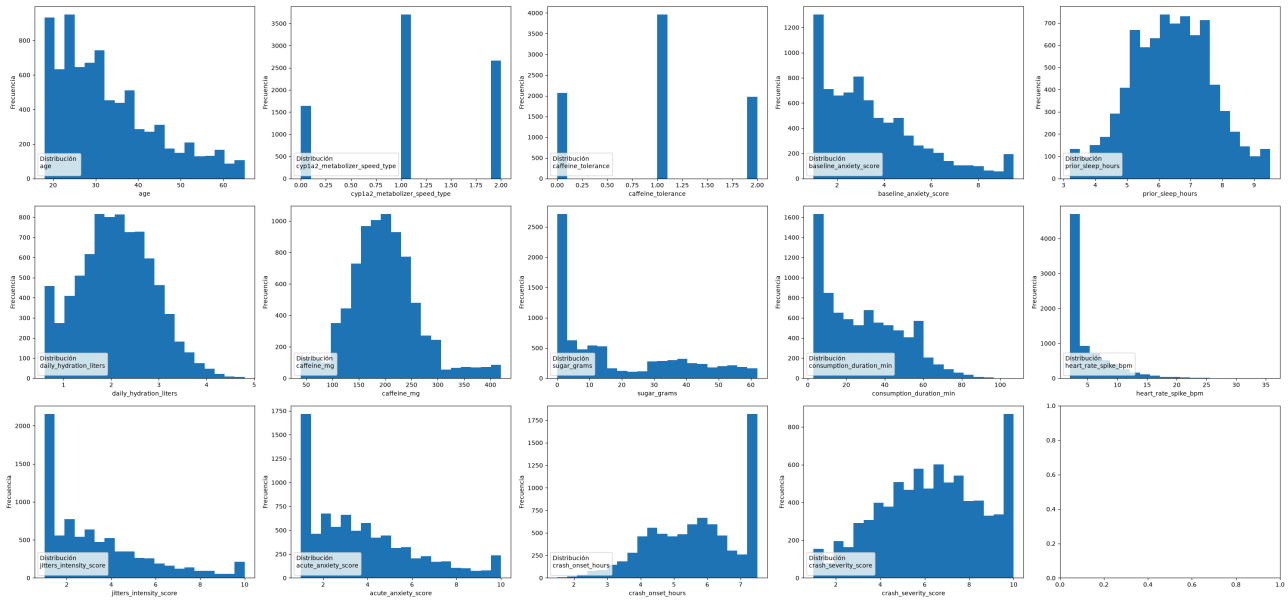
num_vert = 0; num_hor = 0
for cols in numeric_cols:

    axs[num_vert,num_hor].hist(df[cols], bins=20)
    axs[num_vert,num_hor].legend( title=f'Distribución \n{cols}', loc='lower left')
    axs[num_vert,num_hor].axes.get_xaxis().set_visible(True)
    axs[num_vert,num_hor].axes.get_yaxis().set_visible(True)
    axs[num_vert,num_hor].set_xlabel(f'{cols}')
    axs[num_vert,num_hor].set_ylabel('Frecuencia')

    num_hor += 1
    if num_vert == 0 and num_hor == 5:
            num_vert = 1
            num_hor = 0

    elif num_vert == 1 and num_hor == 5:
            num_vert = 2
            num_hor = 0

plt.tight_layout()
fig.show()
```



Así a vista podemos ver que as nosas variables distribúense en xeral nalgunha clase de distribución exponencial ou gaussiana. Na gran maioría o dato da moda resulta atípico, aínda que realmente podemos pensar que isto é realmente un punto que nos indica que as variables están fortemente ligadas, e.g., que moita xente teña un consumo de azucre próximo a 0 gramos pode xustificar que a maioría dos accidentes ocorran máis de 6 horas despois da inxesta se é que o azucre funciona como un catalizador; ou, máis obvio,

que a gran maioría de persoas non se autoperciban síntomas de tremores pode coincidir con non detectar unha Δ BPM tamén na gran maioría. Nos casos gaussianos atopamos unha distribución interesante sobre hábitos de vida como a cantidade de auga que se bebe como as de horas que durmen. Tamén nos decatamos que a gran maioría de pacentes foron precisamente xente xove (menores de 34 anos) aos que se asocia un maior número de consumo de bebidas enerxéticas fronte a formas tradicionais de consumo de cafeína (café ou té).

```
# Pode ser interesante revisar a media e a moda dalgunhas distribucións, por exemplo:
distrib = ('prior_sleep_hours', 'daily_hydration_liters', 'caffeine_mg'
,'crash_onset_hours', 'crash_severity_score')
for cat in distrib:
    print(f'En {cat} a media é {df[cat].mean():.2f} e a moda é
{df[cat].mode().tolist()[0]}')
```

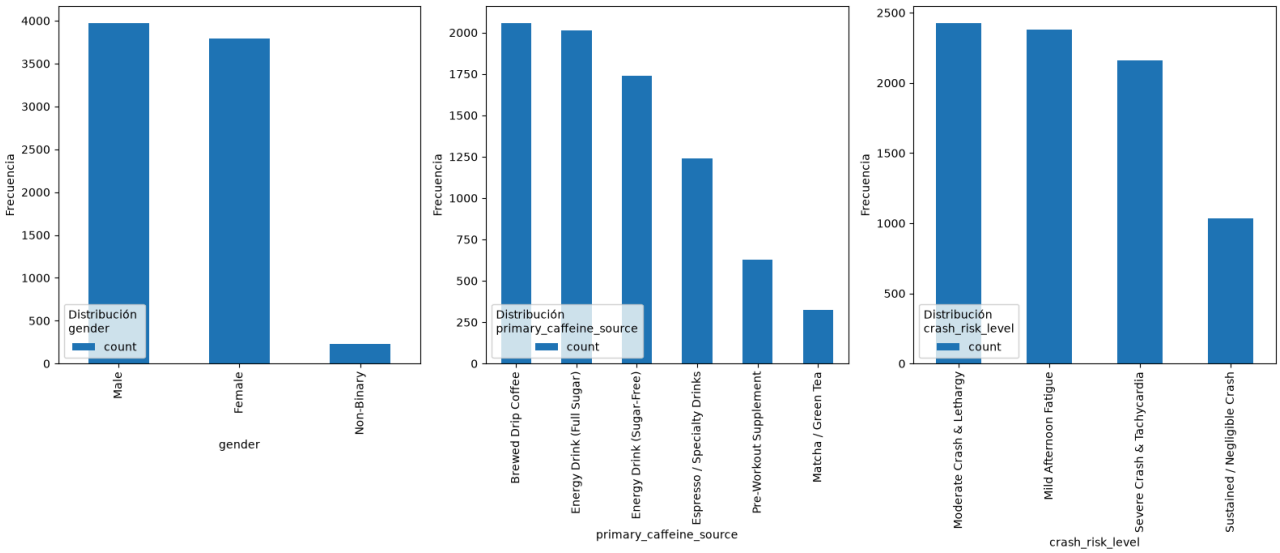
```
En prior_sleep_hours a media é 6.38 e a moda é 6.7
En daily_hydration_liters a media é 2.11 e a moda é 0.6
En caffeine_mg a media é 198.04 e a moda é 197
En crash_onset_hours a media é 5.65 e a moda é 7.5
En crash_severity_score a media é 6.29 e a moda é 10.0
```

```
categorical_cols = df.select_dtypes(include=['object']).columns

fig, axs = plt.subplots(1,3, figsize=(16, 7))

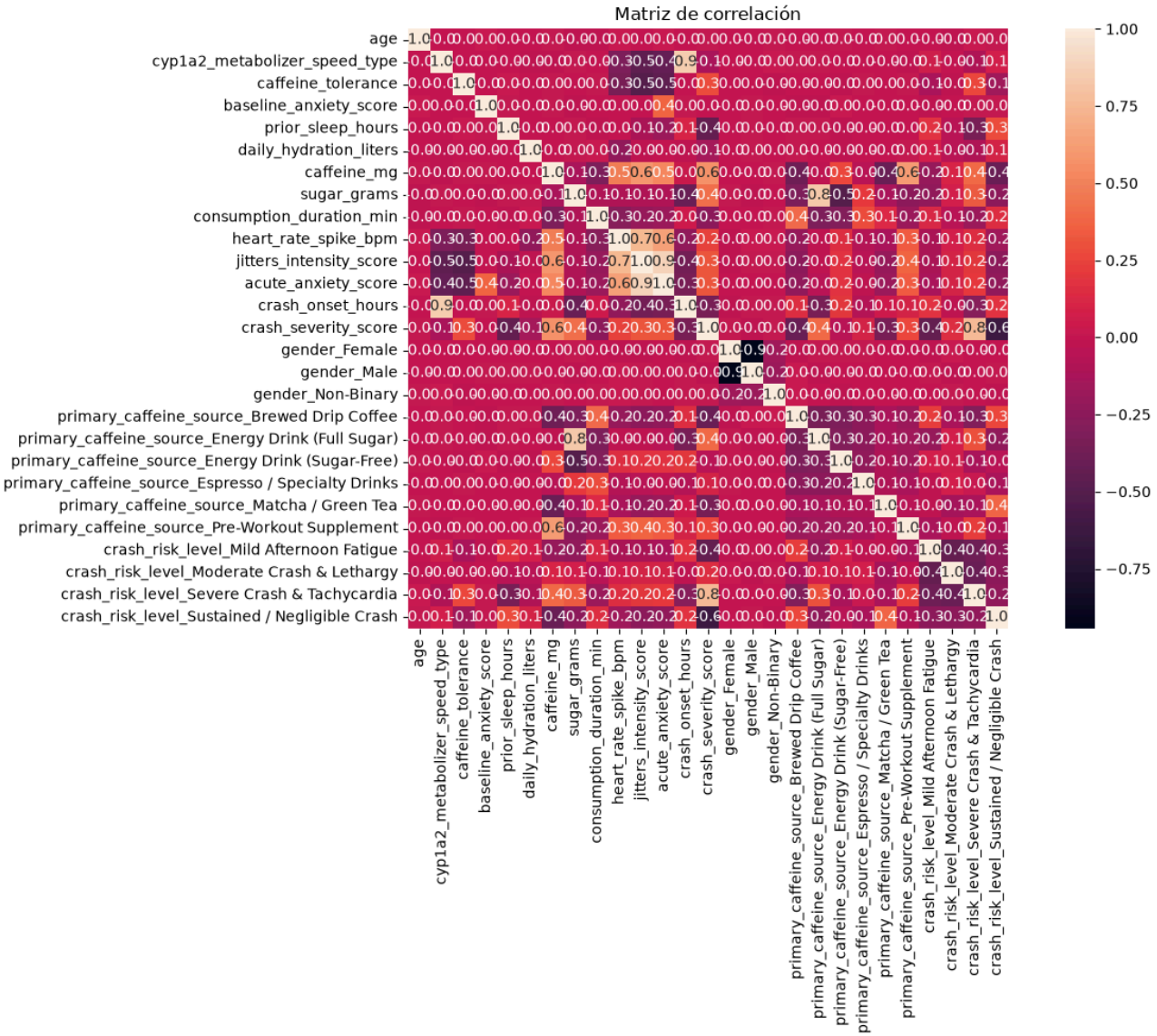
num_hor = 0
for col in categorical_cols[1:]:
    df[col].value_counts().plot(kind='bar', ax=axs[num_hor])
    axs[num_hor].legend(title=f'Distribución \n{col}', loc='lower left')
    axs[num_hor].set_xlabel(col)
    axs[num_hor].set_ylabel('Frecuencia')
    num_hor += 1

plt.tight_layout()
fig.show()
```



```
# Aquí empregamos o DataFrame Df con todas as columnas str traducidas ou empregando
os dummies respectivos; isto é necesario
# para facer correlacións numéricas.

# Mapa de calor da matriz de correlación do noso dataframe completamente numérico
plt.figure(figsize=(14, 10))
sns.heatmap(pd.concat([Df.drop(columns=
['crash_risk_level']),crash_risk_level_dummies],axis=1).select_dtypes(include='number
').corr(),
            annot=True, fmt='.1f', square=True#, cmap='coolwarm'
            )
plt.title('Matriz de correlación')
plt.tight_layout()
plt.show()
```



Vemos que na diagonal aparecen 3 cadradados pequenos de correlación negativa moi significativa, isto derívase de que son as submatrices das variables dummies que son autoexcluíntes. Obviamente non pode haber un suxeito home e muller á vez (obviando identificacións de xénero fluído) ou que alguén teña dúas gravidades distintas no crash.

Vemos a continuación un pouco máis en detalle algunha variable significativa nas predicións.

```
# Imos facer categorías para "caffeine_mg" e "sugar_grams" para comparalas con
"crash_risk_level"
# As categorías escolloas así un pouco a ollo, a cafeína semella unha gaussiana así
facemos division 25 50 25
# e o azucre é exponencial ou incluso cadrada se obviamos o 0, escollemos unha
equiseparación 33.33 33.33 33.33
# Tamén compararemos as "primary_caffeine_source" con "crash_risk_level"

p25, p75 = df["caffeine_mg"].quantile([0.25, 0.75])
p33, p66 = df["sugar_grams"].quantile([0.3333, 0.6666])

def categorize_caffeine(score): # en cuartiles (aínda que o primer 25 chega aos 157mg
que me parece pila)
    # corrixome unha enerxética zero do Día (a que me estou tomando agora mesmo)
pequena (250ml) ten 31mg/100ml ⟶ 77,5 mg
    # logo, como a maioría de enerxéticas son de 500 ml, cunha cantidade semellante
de cafeína ten uns 150 mg
    if score ⩽ p25: # <25%
        return 'Low'
    elif score ⩽ p75: # 25 - 75%
            return 'Medium'
    else:
        return 'High' # >75%

def categorize_sugar(score): # en terciles
    if score ⩽ p33: # <33.33%
        return 'Low'
    elif score ⩽ p66:
            return 'Medium' # 33.33-66.66%
    else:
        return 'High' # >66.66%

caffeine_level = df['caffeine_mg'].apply(categorize_caffeine)
sugar_level = df['sugar_grams'].apply(categorize_sugar)

corr_cafe_sugar_source = pd.concat([caffeine_level, sugar_level,
df['primary_caffeine_source'],df['crash_risk_level']], axis=1)
corr_cafe_sugar_source.head()
```

	caffeine_mg	sugar_grams	primary_caffeine_source	crash_risk_level
0	Low	Medium	Brewed Drip Coffee	Mild Afternoon Fatigue
1	Medium	Low	Energy Drink (Sugar-Free)	Sustained / Negligible Crash
2	Medium	High	Energy Drink (Full Sugar)	Moderate Crash & Lethargy
3	High	Low	Pre-Workout Supplement	Severe Crash & Tachycardia

	caffeine_mg	sugar_grams	primary_caffeine_source	crash_risk_level
4	Medium	High	Energy Drink (Full Sugar)	Severe Crash & Tachycardia

```
Scores = ('caffeine_mg', 'sugar_grams')
Rangos = ('Low', 'Medium', 'High')
cores = ['turquoise', 'orchid', 'r']

fig, axs = plt.subplots(1, 3, figsize=(14, 6))

for i, score in enumerate(Scores):
    score_distribution = (
        corr_cafe_sugar_source
        .groupby('crash_risk_level')[score]
        .value_counts(normalize=True)
        .unstack()
        .reindex(columns=Rangos, fill_value=0)
        )

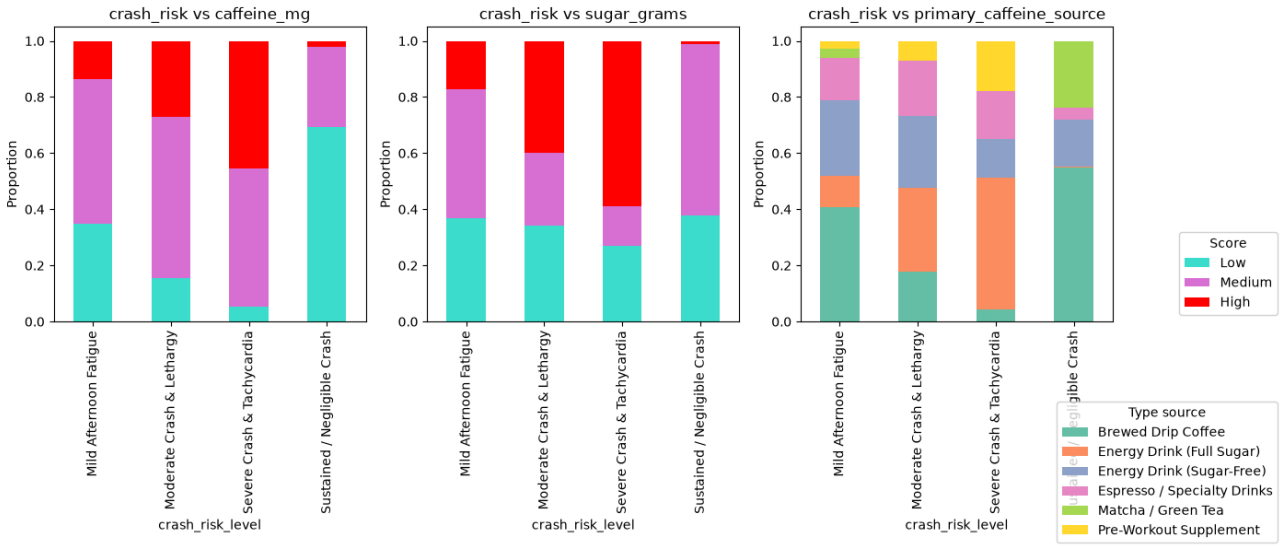
    score_distribution.plot(
        kind='bar',
        stacked=True,
        ax=axs[i],
        color=cores,
        legend=False
        )

    axs[i].set_title(f'crash_risk vs {score}')
    # axs[i].set_xlabel('Crash risk level') # non creo que falla falta, é bsatante
intuitiva a gráfica
    axs[i].set_ylabel('Proportion')

# facemos un plot especial para poder ver stackeadas dúas variables str por
proporción
crash_vs_source = pd.crosstab(corr_cafe_sugar_source['crash_risk_level'],
corr_cafe_sugar_source['primary_caffeine_source'], normalize='index')
cores = sns.color_palette('Set2', n_colors=crash_vs_source.shape[1])
crash_vs_source.plot(kind='bar', stacked=True, ax=axs[2], color=cores, legend=False)
axs[2].set_title(f'crash_risk vs primary_caffeine_source')
axs[2].set_ylabel('Proportion')

handles, label_1 = axs[0].get_legend_handles_labels()
handles1, label_2 = axs[2].get_legend_handles_labels()
fig.legend(handles, label_1, title='Score', loc='center right')
fig.legend(handles1, label_2, title='Type source', loc='lower right')

plt.tight_layout(rect=[0, 0, 0.88, 1])
plt.show()
```



Agora, previo a proceder ás dúas análises de predicción da variable "crash_risk_level" debemos separar as nosas mostras de datos en datos de entrenamento e de testeo. Será necesario que utilizemos o dataframe onde substituímos as columnas str polos dummies respectivos (Df) que serán sempre os datos de interese no resto do noso estudo.

```
y1 = Df.crash_risk_level

labels_1 = ['Mild Afternoon Fatigue', # son os elementos de y1, será necesario telo definido. De momento non sei se fará falta un labels_2
    'Moderate Crash & Lethargy',
    'Severe Crash & Tachycardia',
    'Sustained / Negligible Crash']
```

```
x1 = Df.drop(columns=['crash_risk_level'])
```

```
split = round(x1.shape[0]*0.80) # todas as X son do mesmo tamaño, non distinguimos, 80% de entreno e 20% de testeo
x1_train, x1_test = x1[:split], x1[split:]
y1_train, y1_test = y1[:split], y1[split:]
```

1.3 3. Análise con sklearn (regresión loxística)

Primeiro normalizo as variables

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

x1_train_scaled = scaler.fit_transform(x1_train)
x1_test_scaled = scaler.transform(x1_test)
```

3.0.1 Modelo Linear

Toca empezar a crear modelos, o primeiro será linear e empregarei un cálculo a man da optimización do hiperparámetro alpha, este é bastante pequeno así que para reducir o costo computacional xa o teño posto cerquiña do valor real que calcula (estiven 12 minutos esperando cunha partición máis fina nun intervalo máis grande XoX)

```
from sklearn.linear_model import SGDClassifier
from sklearn.model_selection import cross_val_score

alpha = np.arange(1e-15, 0.000001, 0.00000005)
val_scores = np.zeros(len(alpha))

for i, a in enumerate(alpha):
    model_cross = SGDClassifier(
            loss='hinge', penalty='l2', alpha=a, random_state=100
        )
    score = cross_val_score(model_cross, x1_train_scaled, y1_train, cv=5)
    val_scores[i] = score.mean()

ind = np.argmax(val_scores)
print('O mellor valor de alpha é', alpha[ind])
```

```
O mellor valor de alpha é 6.500000009999999e-07
```

```
model_linear = SGDClassifier(loss='hinge', penalty='l2', alpha=alpha[ind])
model_trained_1 = model_linear.fit(x1_train_scaled, y1_train)

y1_hat_train_1 = model_trained_1.predict(x1_train_scaled)
y1_hat_test_1 = model_trained_1.predict(x1_test_scaled)
```

```
from sklearn.metrics import r2_score
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report

print(f'A exactitude de predición do modelo linear no conxunto de adestramento é
{accuracy_score(y1_train, y1_hat_train_1):.4f}')
print(f'A exactitude de predición do modelo linear no conxunto de teste é
{accuracy_score(y1_test, y1_hat_test_1):.4f}')

print(
    classification_report(
        y1_test,
        y1_hat_test_1,
        target_names=labels_1
    )
)
```

```
A exactitude de predición do modelo linear no conxunto de adestramento é 0.7331
A exactitude de predición do modelo linear no conxunto de teste é 0.7044
                            precision    recall  f1-score   support

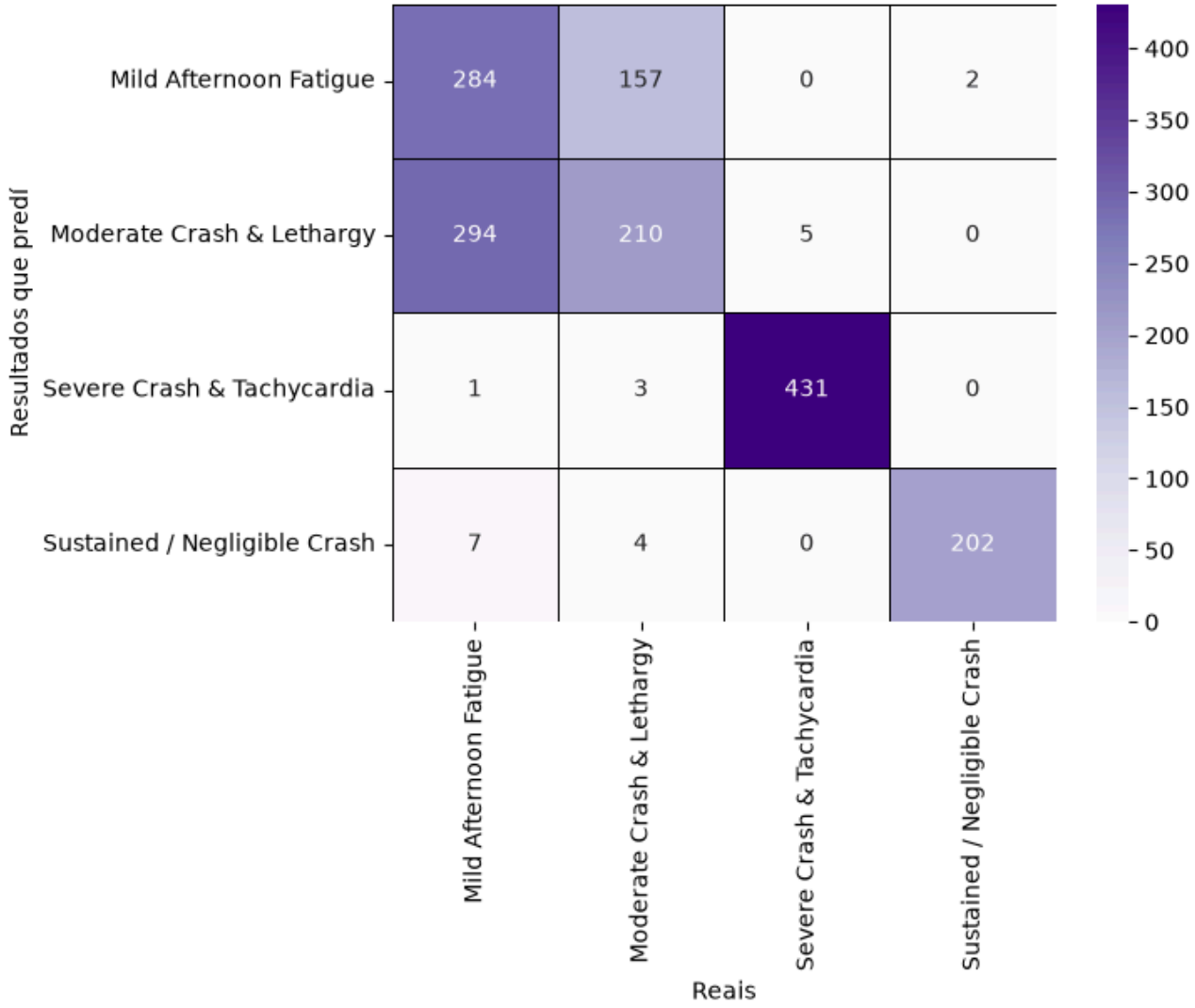
      Mild Afternoon Fatigue       0.48      0.64      0.55       443
   Moderate Crash & Lethargy       0.56      0.41      0.48       509
  Severe Crash & Tachycardia       0.99      0.99      0.99       435
Sustained / Negligible Crash       0.99      0.95      0.97       213

                    accuracy                           0.70      1600
                   macro avg       0.76      0.75      0.75      1600
                weighted avg       0.71      0.70      0.70      1600
```

```
from sklearn.metrics import confusion_matrix

cm_1 = confusion_matrix(y1_test, y1_hat_test_1, labels=labels_1)

sns.heatmap(cm_1, cmap='Purples', annot=True, fmt='d', linewidths=0.5,
linecolor='black', xticklabels=labels_1, yticklabels=labels_1) #Graficamos la matriz
de confusión en escala de colores violeta, hacemos que aparezcan los valores de los
resultados (en forma de números enteros) y sus clases y que la separación de los
resultados sea una línea negra
plt.ylabel('Resultados que predí') #Ponemos título al eje y
plt.xlabel('Reais') #Ponemos título al eje x
plt.show() #Hacemos que se muestre el mapa de calor
```



3.0.2 Modelo RBF

Modelo Radial Basis Function, é un modelo gaussiano con fronteiras non-lineares, espérase que se axuste moito mellor que unha recta ao considerar precisamente asociacións non lineares

```
from sklearn.svm import SVC

model_rbf = SVC(
    kernel='rbf',
    C=1.0,
    gamma='scale'
)

model_trained_2 = model_rbf.fit(x1_train_scaled, y1_train)

y1_hat_train_2 = model_trained_2.predict(x1_train_scaled)
y1_hat_test_2 = model_trained_2.predict(x1_test_scaled)
```

```
print(f'A exactitude de predición do modelo lin no conxunto de adestramento é
{accuracy_score(y1_train, y1_hat_train_2):.4f}')
print(f'A exactitude de predición do modelo lin no conxunto de teste é
{accuracy_score(y1_test, y1_hat_test_2):.4f}')

print(
    classification_report(
        y1_test,
        y1_hat_test_2,
        target_names=labels_1
    )
)
```

```
A exactitude de predición do modelo lin no conxunto de adestramento é 0.9608
A exactitude de predición do modelo lin no conxunto de teste é 0.9244
                            precision    recall  f1-score   support

      Mild Afternoon Fatigue       0.90      0.94      0.92       443
   Moderate Crash & Lethargy       0.91      0.94      0.92       509
  Severe Crash & Tachycardia       0.96      0.92      0.94       435
Sustained / Negligible Crash       0.94      0.86      0.90       213

                    accuracy                           0.92      1600
                   macro avg       0.93      0.92      0.92      1600
                weighted avg       0.93      0.92      0.92      1600
```

```
cm_2 = confusion_matrix(y1_test, y1_hat_test_2, labels=labels_1)

sns.heatmap(cm_2, cmap='Purples', annot=True, fmt='d', linewidths=0.5,
linecolor='black', xticklabels=labels_1, yticklabels=labels_1)
plt.ylabel('Resultados que predí')
plt.xlabel('Reais')
plt.show()
```



1.3.1 3.1 Análise sen sesgo

```
# o nome da sección é por poñer algo, non sei que poñer.
from sklearn.feature_selection import mutual_info_classif

mi = mutual_info_classif(
    x1_train_scaled,
    y1_train,
    random_state=100
)

for col, val in sorted(zip(x1.columns, mi),
                       key=lambda x1: x1[1],
                       reverse=True):
    print(col, round(val,4))
```

```
crash_severity_score 1.3402
caffeine_mg 0.217
sugar_grams 0.1812
primary_caffeine_source_Brewed Drip Coffee 0.0941
prior_sleep_hours 0.0891
primary_caffeine_source_Energy Drink (Full Sugar) 0.0816
crash_onset_hours 0.0753
jitters_intensity_score 0.0701
primary_caffeine_source_Matcha / Green Tea 0.0641
acute_anxiety_score 0.0559
caffeine_tolerance 0.0411
heart_rate_spike_bpm 0.0381
consumption_duration_min 0.0333
primary_caffeine_source_Pre-Workout Supplement 0.0331
primary_caffeine_source_Espresso / Specialty Drinks 0.0271
cyp1a2_metabolizer_speed_type 0.0227
primary_caffeine_source_Energy Drink (Sugar-Free) 0.0196
daily_hydration_liters 0.0088
baseline_anxiety_score 0.0071
gender_Male 0.0049
gender_Non-Binary 0.0002
age 0.0
gender_Female 0.0
```

aquí vemos que a columna "crash_severity_score" ten un 1.34 de correlación o que quere dicir que lle está "chivando" (que é ao que lle chamo sesgo) moita información ao modelo, o seu sería ver como se comporta sen empregar esta columna, logo, repetindo considerando ese caso, i.e., facendo un drop no noso dataframe definindo de novo as x's e as y's

```
y2 = Df.crash_risk_level # satisfaise que y1 == y2; pero por poder facelos
distinguibles
x2 = Df.drop(columns=['crash_risk_level','crash_severity_score'])

split = round(x2.shape[0]*0.80) # todas as X son do mesmo tamaño, non distinguimos,
80% de entreno e 20% de testeo
x2_train, x2_test = x2[:split], x2[split:]
y2_train, y2_test = y2[:split], y2[split:]

x2_train_scaled = scaler.fit_transform(x2_train)
x2_test_scaled = scaler.transform(x2_test)
```

3.1.1 Estimación, training e resultados do Modelo Linear/Loxístico linear

Agora imos predecir adecuadamente os parámetros de todos os nosos modelos. En vez de empregar un cálculo a man como se fixo antes para o caso linear empregamos unha función de sklearn que se chama `GridSearchCV`. Primeiro vemos o caso da linear a ver que tal se axusta e de ser necesario correximos o `loss`, que antes usamos `loss = hinge` que é o caso linear, e o `penalty` que os escollimos anteriormente sen xustificación. É de esperar que o axuste loxístico (`loss = log_loss`) sexa mellor que linear, así que comparamos o valor do `best_score_` para determinalo; representaremos só os resultados do mellor entre os dous.

```
from sklearn.model_selection import GridSearchCV

param_linear = {
    'loss': ['hinge'],
    'penalty': ['l2', 'l1', 'elasticnet'],
    'alpha': [1e-5, 1e-4, 1e-3, 1e-2, 5e-3]
}

grid_linear = GridSearchCV(
    estimator = SGDClassifier(random_state=100),
    param_grid = param_linear,
    cv = 10, # é o número de cross validations que fai o algoritmo; se poño moitas
posibilidades no param_grid é necesario aumentalo, pero aumenta moitisimo o custe
computacional
    scoring = 'f1_macro', # neste caso a columna "crash_risk_level" ten as
proporcións de datos 2377, 1036, 2426, 2161 ⟶ 29.7, 12.9, 30.3, 27.0 %
    # non é unha diferenza moi significativa polo que tanto empregar 'accuracy' como
'f1_macro' son boas opcións; escollo a segunda para balancear mellor
    n_jobs = -1 # emprega todos os núcleos da cpu: moito máis rápido pero pode
saturar outros programas abertos
)

grid_linear.fit(x2_train_scaled, y2_train)
best_model_linear = grid_linear.best_estimator_ # escollemos automaticamente o modelo
mellor estimado (o de mellores parámetros)
```

```
param_log = {
    'loss': ['log_loss'], # usar 'modified_huber' dá resultados moi lixeiramente
mellores, pero non é tipo loxístico
    'penalty': ['l2', 'l1', 'elasticnet'],
    'alpha': [1e-5, 1e-4, 1e-3, 1e-2, 5e-3]
}

grid_log = GridSearchCV(
    estimator = SGDClassifier(random_state=100),
    param_grid = param_log,
    cv = 10,
    scoring = 'f1_macro',
    n_jobs = -1
)

grid_linear.fit(x2_train_scaled, y2_train)
grid_log.fit(x2_train_scaled, y2_train)
best_model_log = grid_log.best_estimator_

# o mellor modelo escóllese precisamente con aquel que entre todas as combinacións de
parámetros dados posibles teña o scoring (f1-macro) máximo,
# cada scoring é media de todas as validacións cruzadas para esos parámetros
print(f'O f1-macro medio das n validacións cruzadas cos mellores parámetros é
{grid_linear.best_score_:.4f}')
print(f'O f1-macro medio das n validacións cruzadas cos mellores parámetros é
{grid_log.best_score_:.4f}')
if grid_log.best_score_ > grid_linear.best_score_:
    print('A regresión loxística axústase mellor que a linear.')
else:
    print('A regresión linear axústase mellor que a loxística.')
```

```
O f1-macro medio das n validacións cruzadas cos mellores parámetros é 0.6697
O f1-macro medio das n validacións cruzadas cos mellores parámetros é 0.7467
A regresión loxística axústase mellor que a linear.
```

```
y2_hat_train_1 = best_model_log.predict(x2_train_scaled)
y2_hat_test_1 = best_model_log.predict(x2_test_scaled)

print(f'Os mellores axustes para o modelo Loxístico son {grid_log.best_params_}')
print(f'A exactitude de predición do modelo Loxístico no conxunto de adestramento é
{accuracy_score(y2_train, y2_hat_train_1):.4f}')
print(f'A exactitude de predición do modelo Loxístico no conxunto de teste é
{accuracy_score(y2_test, y2_hat_test_1):.4f}\n')

print(
    classification_report(
        y2_test,
        y2_hat_test_1,
        target_names=labels_1
    )
)
```

```
Os mellores axustes para o modelo Loxístico son {'alpha': 0.005, 'loss': 'log_loss',
'penalty': 'l1'}
A exactitude de predición do modelo Loxístico no conxunto de adestramento é 0.7439
A exactitude de predición do modelo Loxístico no conxunto de teste é 0.7331

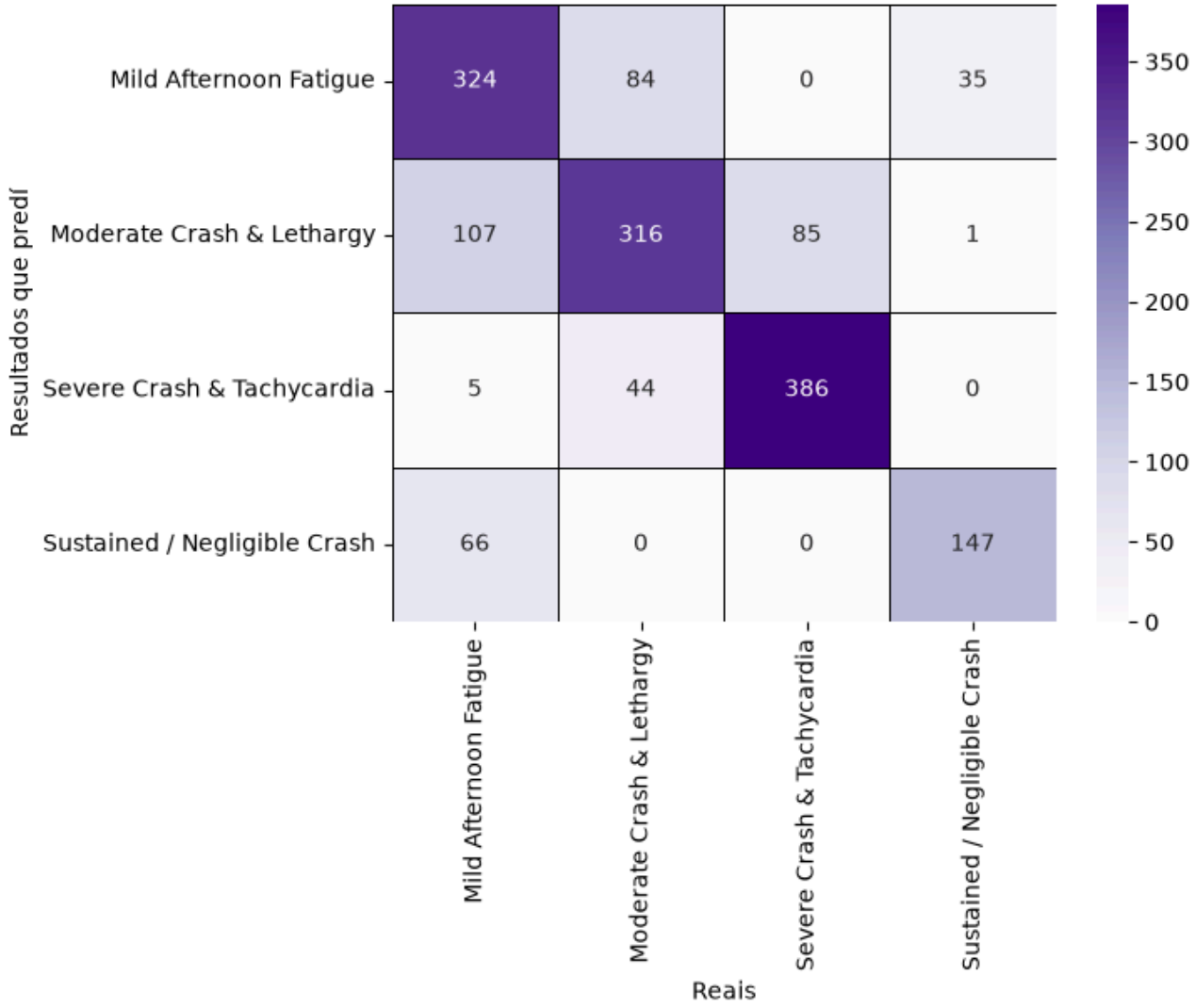
                           precision    recall  f1-score   support

      Mild Afternoon Fatigue       0.65      0.73      0.69       443
   Moderate Crash & Lethargy       0.71      0.62      0.66       509
  Severe Crash & Tachycardia       0.82      0.89      0.85       435
Sustained / Negligible Crash       0.80      0.69      0.74       213

                    accuracy                           0.73      1600
                   macro avg       0.74      0.73      0.74      1600
                weighted avg       0.73      0.73      0.73      1600
```

```
cm_3 = confusion_matrix(y2_test, y2_hat_test_1, labels=labels_1)

sns.heatmap(cm_3, cmap='Purples', annot=True, fmt='d', linewidths=0.6,
linecolor='black', xticklabels=labels_1, yticklabels=labels_1)
plt.ylabel('Resultados que predí')
plt.xlabel('Reais')
plt.show()
```



3.1.2 Estimación, training e resultados do Modelo RBF

```
param_rbf = {
    'C': [1, 10, 20, 50],
    'gamma': [0.005, 0.01, 0.05, 0.1]
}

grid_rbf = GridSearchCV(
    estimator = SVC(kernel='rbf'),
    param_grid = param_rbf,
    cv = 5,
    scoring = 'f1_macro',
    n_jobs = -1
)

grid_rbf.fit(x2_train_scaled, y2_train)
best_model_rbf = grid_rbf.best_estimator_

print(f'O f1-macro medio das n validacións cruzadas cos mellores parámetros é
{grid_rbf.best_score_:.4f}')
```

```
O f1-macro medio das n validacións cruzadas cos mellores parámetros é 0.8039
```

```
y2_hat_train_2 = best_model_rbf.predict(x2_train_scaled)
y2_hat_test_2 = best_model_rbf.predict(x2_test_scaled)

print(f'Os mellores axustes para o modelo RBF son {grid_rbf.best_params_}')
print(f'A exactitude de predición do modelo RBF no conxunto de adestramento é
{accuracy_score(y2_train, y2_hat_train_2):.4f}')
print(f'A exactitude de predición do modelo RBF no conxunto de teste é
{accuracy_score(y2_test, y2_hat_test_2):.4f}\n')

print(
    classification_report(
        y2_test,
        y2_hat_test_2,
        target_names=labels_1
    )
)
```

```
Os mellores axustes para o modelo RBF son {'C': 20, 'gamma': 0.01}
A exactitude de predición do modelo RBF no conxunto de adestramento é 0.8384
A exactitude de predición do modelo RBF no conxunto de teste é 0.7875

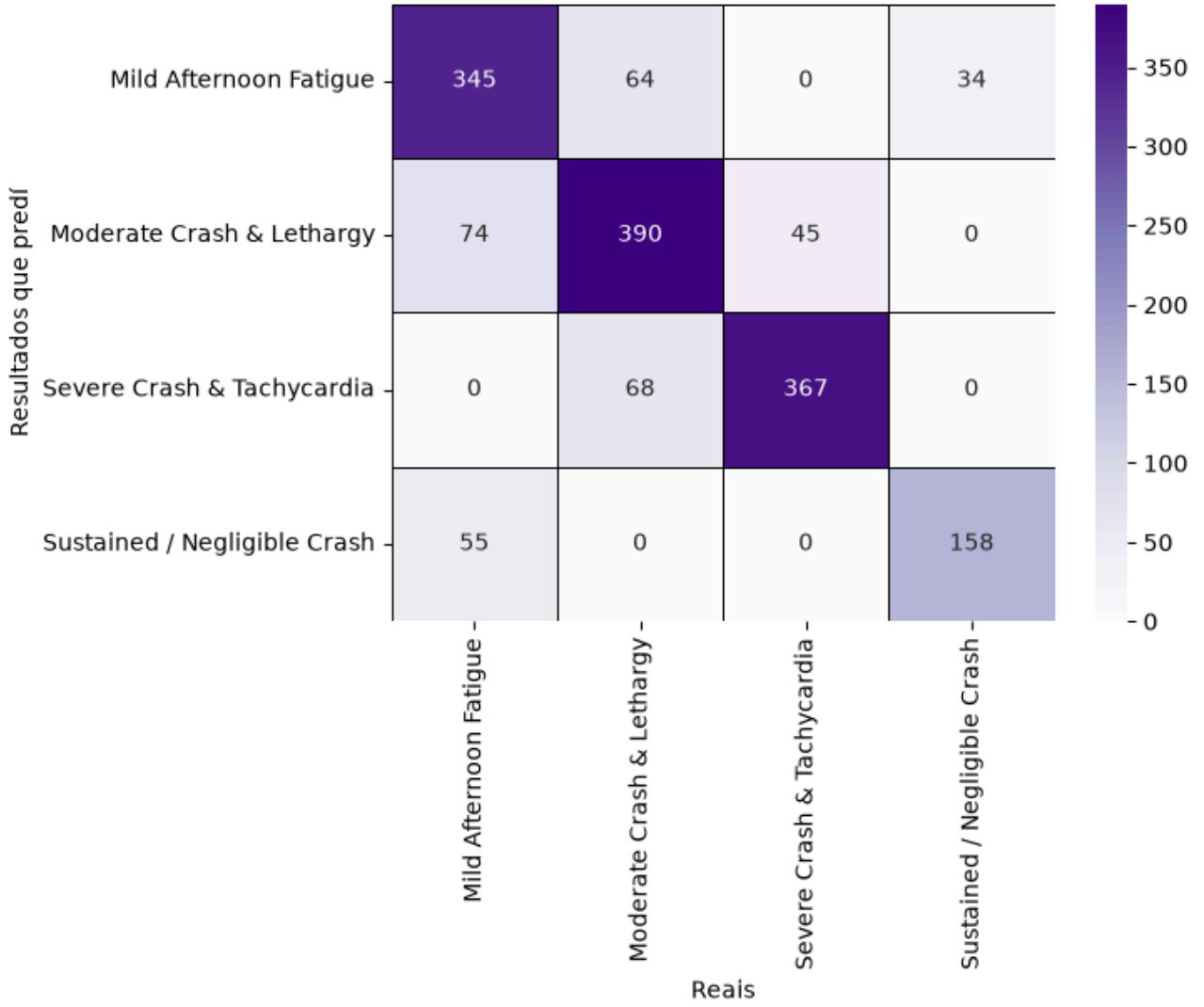
                           precision    recall  f1-score   support

      Mild Afternoon Fatigue       0.73      0.78      0.75       443
   Moderate Crash & Lethargy       0.75      0.77      0.76       509
  Severe Crash & Tachycardia       0.89      0.84      0.87       435
Sustained / Negligible Crash       0.82      0.74      0.78       213

                    accuracy                           0.79      1600
                   macro avg       0.80      0.78      0.79      1600
                weighted avg       0.79      0.79      0.79      1600
```

```
cm_4 = confusion_matrix(y2_test, y2_hat_test_2, labels=labels_1)

sns.heatmap(cm_4, cmap='Purples', annot=True, fmt='d', linewidths=0.6,
linecolor='black', xticklabels=labels_1, yticklabels=labels_1)
plt.ylabel('Resultados que predí')
plt.xlabel('Reais')
plt.show()
```



3.1.3 Estimación, training e resultados do Modelo RandomForest

Imos probar ademais tamén con outros modelos, neste caso tipo árbore. Por unha banda empregamos un sistema de árbores independentes `RandomForestClassifier()` e pola outra un sistema de árbores secuenciais `HistGradientBoostingClassifier`

Caption. O tempo de compilado dos modelos tipo árbore é bastante extenso, teño un procesador bastante potente e aínda así tárdame ao redor 1 minuto cada un.

```
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier

param_forest = {
    'n_estimators': [400, 500, 600],
    'max_depth': [None, 10, 20, 25],
    'min_samples_leaf': [1, 2, 3]
}

grid_forest = GridSearchCV(
    RandomForestClassifier(random_state=100),
    param_grid = param_forest,
    cv = 5,
    scoring = 'f1_macro',
    n_jobs = -1
)

grid_forest.fit(x2_train_scaled, y2_train)
best_model_forest = grid_forest.best_estimator_

print(f'O f1-macro medio das n validacións cruzadas cos mellores parámetros é
{grid_forest.best_score_:.4f}')
```

```
O f1-macro medio das n validacións cruzadas cos mellores parámetros é 0.7522
```

```
y2_hat_train_3 = best_model_forest.predict(x2_train_scaled)
y2_hat_test_3 = best_model_forest.predict(x2_test_scaled)

print(f'Os mellores axustes para o modelo RandomForestClassifier son
{grid_forest.best_params_}')
print(f'A exactitude de predición do modelo RandomForestClassifier no conxunto de
adestramento é {accuracy_score(y2_train, y2_hat_train_3):.4f}')
print(f'A exactitude de predición do modelo RandomForestClassifier no conxunto de
teste é {accuracy_score(y2_test, y2_hat_test_3):.4f}\n')

print(
    classification_report(
        y1_test,
        y1_hat_test_2,
        target_names=labels_1
    )
)
```

```
Os mellores axustes para o modelo RandomForestClassifier son {'max_depth': 25,
'min_samples_leaf': 1, 'n_estimators': 500}
A exactitude de predición do modelo RandomForestClassifier no conxunto de adestramento
é 1.0000
A exactitude de predición do modelo RandomForestClassifier no conxunto de teste é
0.7600

                           precision    recall  f1-score   support

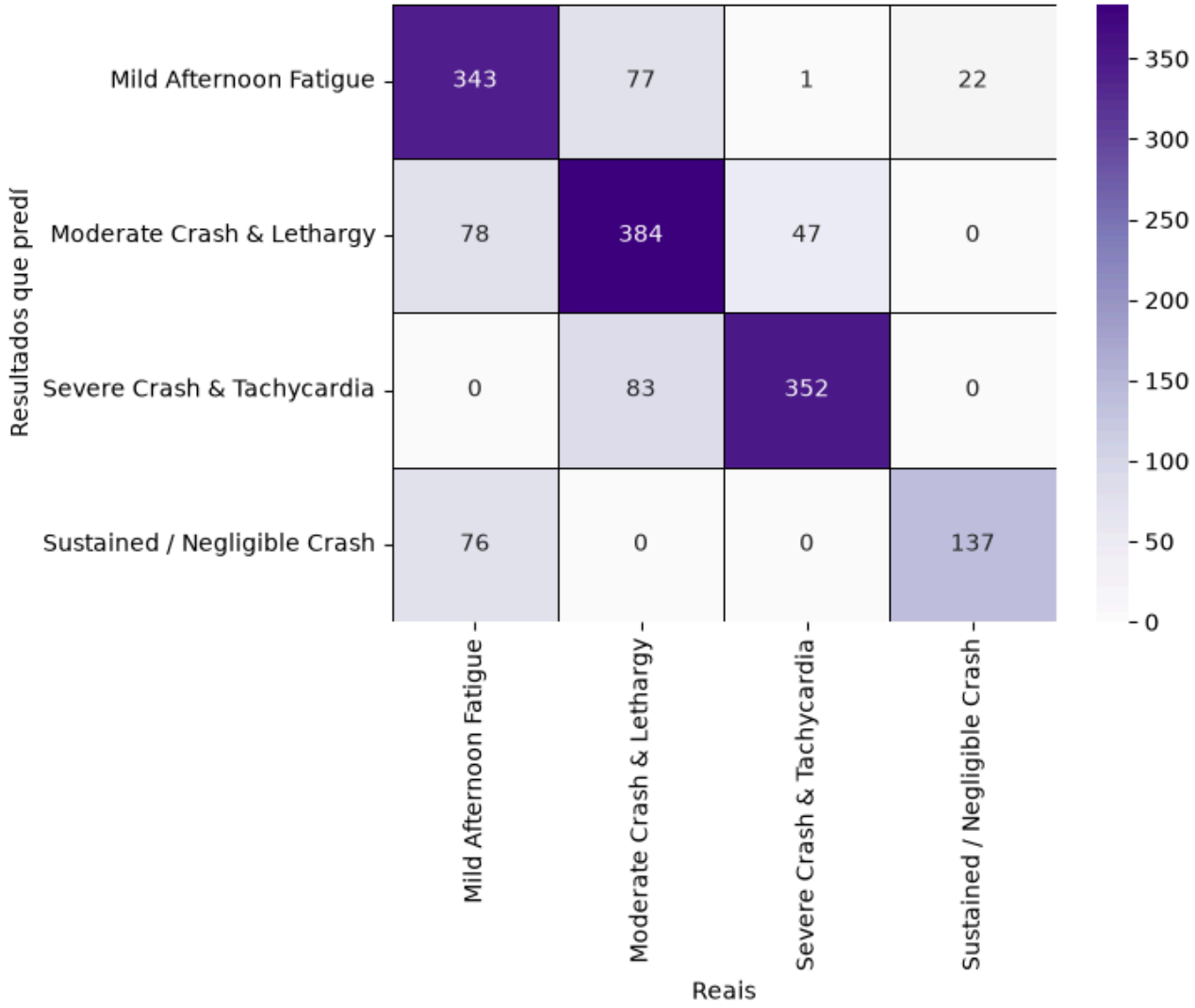
      Mild Afternoon Fatigue       0.90      0.94      0.92       443
   Moderate Crash & Lethargy       0.91      0.94      0.92       509
  Severe Crash & Tachycardia       0.96      0.92      0.94       435
Sustained / Negligible Crash       0.94      0.86      0.90       213

                    accuracy                           0.92      1600
                   macro avg       0.93      0.92      0.92      1600
                weighted avg       0.93      0.92      0.92      1600
```

As nosas árbores teñen un n bastante alto de estimadores (500) o que lle permite practicamente "memorizar" os resultados do entrenamento, de aí que a precisión á hora de predecilos sexa exacta e sen erros

```
cm_5 = confusion_matrix(y2_test, y2_hat_test_3, labels=labels_1)

sns.heatmap(cm_5, cmap='Purples', annot=True, fmt='d', linewidths=0.5,
linecolor='black', xticklabels=labels_1, yticklabels=labels_1)
plt.ylabel('Resultados que predí')
plt.xlabel('Reais')
plt.show()
```



3.1.4 Estimación, training e resultados do Modelo HistGradientBoosting

```
param_boost = {
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 5, 10],
    'max_iter': [100, 200, 500]
}

grid_boost = GridSearchCV(
    HistGradientBoostingClassifier(random_state=100),
    param_grid = param_boost,
    cv = 5,
    scoring = 'f1_macro',
    n_jobs = -1
)

grid_boost.fit(x2_train_scaled, y2_train)
best_model_boost = grid_boost.best_estimator_

print(f'O f1-macro medio das n validacións cruzadas cos mellores parámetros é
{grid_boost.best_score_:.4f}')
```

```
O f1-macro medio das n validacións cruzadas cos mellores parámetros é 0.7866
```

```
y2_hat_train_4 = best_model_boost.predict(x2_train_scaled)
y2_hat_test_4 = best_model_boost.predict(x2_test_scaled)

print(f'Os mellores axustes para o modelo HistGradientBoosting son
{grid_boost.best_params_}')
print(f'A exactitude de predición do modelo HistGradientBoosting no conxunto de
adestramento é {accuracy_score(y2_train, y2_hat_train_4):.4f}')
print(f'A exactitude de predición do modelo HistGradientBoosting no conxunto de teste
é {accuracy_score(y2_test, y2_hat_test_4):.4f}\n')

print(
    classification_report(
        y2_test,
        y2_hat_test_4,
        target_names=labels_1
    )
)
```

```
Os mellores axustes para o modelo HistGradientBoosting son {'learning_rate': 0.05,
'max_depth': 3, 'max_iter': 500}
A exactitude de predición do modelo HistGradientBoosting no conxunto de adestramento é
0.9136
A exactitude de predición do modelo HistGradientBoosting no conxunto de teste é 0.7750

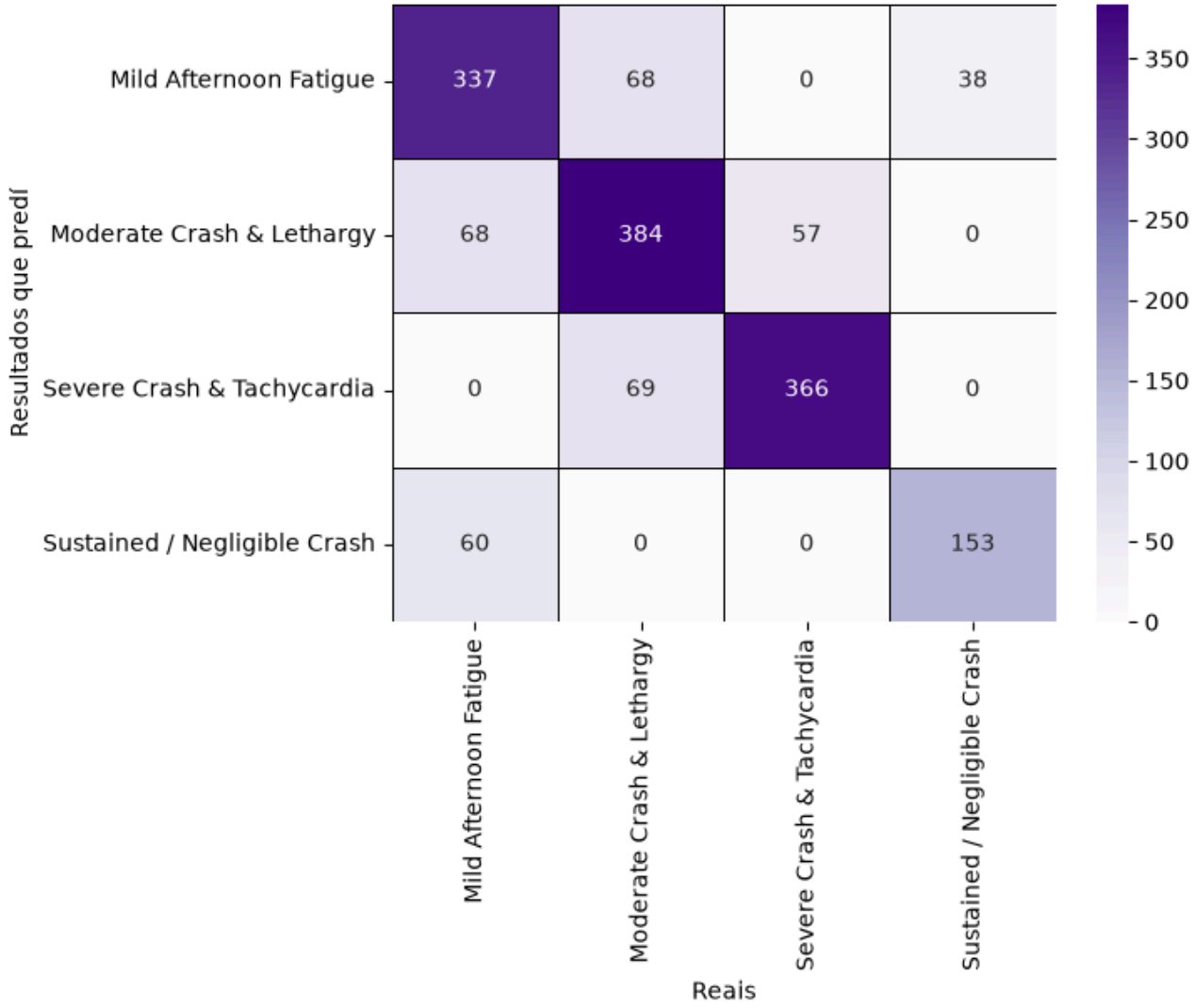
                           precision    recall  f1-score   support

      Mild Afternoon Fatigue       0.72      0.76      0.74       443
   Moderate Crash & Lethargy       0.74      0.75      0.75       509
  Severe Crash & Tachycardia       0.87      0.84      0.85       435
Sustained / Negligible Crash       0.80      0.72      0.76       213

                    accuracy                           0.78      1600
                   macro avg       0.78      0.77      0.77      1600
                weighted avg       0.78      0.78      0.78      1600
```

```
cm_6 = confusion_matrix(y2_test, y2_hat_test_4, labels=labels_1)

sns.heatmap(cm_6, cmap='Purples', annot=True, fmt='d', linewidths=0.6,
linecolor='black', xticklabels=labels_1, yticklabels=labels_1)
plt.ylabel('Resultados que predí')
plt.xlabel('Reais')
plt.show()
```



1.3.2 3.2 Conclusións

Primeiro fixemos os cálculos incluíndo a columna de score e obtivemos

```
print(f'Para o modelo linear obtivemos unha exactitude do {accuracy_score(y1_test,
y1_hat_test_1)*100:.2f}% e para o RBF unha do {accuracy_score(y1_test,
y1_hat_test_2)*100:.2f}%')
if accuracy_score(y1_test, y1_hat_test_1) > accuracy_score(y1_test, y1_hat_test_2):
    print('A pesar das limitacións dun modelo linear predice mellor que un
gaussiano')
else:
    print('O modelo gaussiano predí mellor, como se esperaba, que o modelo linear')
```

```
Para o modelo linear obtivemos unha exactitude do 70.44% e para o RBF unha do 92.44%
O modelo gaussiano predí mellor, como se esperaba, que o modelo linear
```

Ambos modelos predicen moi ben o conxunto de teste. O linear, como é de esperar, é bastante limitado, pola contra o RBF cun score do 92% é algo extraordinario que se pode xustificar, como se dixo, pola alta correlación entre as columnas "crash_risk_level" e "crash_severity_score".

En consecuencia volvemos a realizar os cálculos eliminando a columna "crash_severity_score" e para asegurar uns bos resultados optimizamos por validación cruzada os parámetros dos modelos. Os nosos resultados de predición sobre o conxunto de teste son

```
print(f'Para o modelo loxístico obtivemos unha exactitude do {accuracy_score(y2_test,
y2_hat_test_1)*100:.2f}%, para o RBF unha do {accuracy_score(y2_test,
y2_hat_test_2)*100:.2f}%, para o modelo RandomForest \nunha do
{accuracy_score(y2_test, y2_hat_test_3)*100:.2f}% e para o modelo
HistGradientBoosting é de {accuracy_score(y2_test, y2_hat_test_4)*100:.2f}%')

resultados = {
    'Loxístico': accuracy_score(y2_test, y2_hat_test_1),
    'RBF': accuracy_score(y2_test, y2_hat_test_2),
    'RandomForest': accuracy_score(y2_test, y2_hat_test_3),
    'HistGradientBoosting': accuracy_score(y2_test, y2_hat_test_4)
}

print(f'Todos os modelos teñen unha capacidade de predición semellante, o
{max(resultados, key=resultados.get)} é lixeiramente mellor, superando incluso aos
modelos de árbore.')
```

```
Para o modelo loxístico obtivemos unha exactitude do 73.31%, para o RBF unha do
78.75%, para o modelo RandomForest
unha do 76.00% e para o modelo HistGradientBoosting é de 77.50%
Todos os modelos teñen unha capacidade de predición semellante, o RBF é lixeiramente
mellor, superando incluso aos modelos de árbore.
```

1.3.3 3.3 Análise de regresións (Score)

O primeiro é definir as variables y coa columna "crash_severity_score", que é numérica (float entre 0 e 10). Non fará falta definir x porque a x2 definida anteriormente é precisamente a que nos interesa.

```
# X é identico a x2_train_scaled, x2_test_scaled
y_score = Df['crash_severity_score']
y_score_train, y_score_test = y_score[:split], y_score[split:]
```

Imos tamén probar un par de modelos como fixemos até agora: un linear, un poly de grao 2, 3, 5 e 10 e finalmente un rbf. Aplicaremos tamén un proceso de cross validation de moi poucos n porque ten un custo computacional enorme (varios minutos).

3.3.1 Estimación, training e resultados do modelo SVR linear

```
from sklearn.svm import SVR
from sklearn.model_selection import KFold
cv = KFold(n_splits=2, shuffle=True, random_state=100)

param_svr_lin = {'C': [1, 10, 100]}

grid_svr_lin = GridSearchCV( # son 2 minutos
    estimator = SVR(kernel='linear'),
    param_grid = param_svr_lin,
    cv = 5,
    scoring = 'r2', # como estamos avaliando un axuste a unha función interésanos o
Pearson
    n_jobs = -1
)

grid_svr_lin.fit(x2_train_scaled, y_score_train)
best_model_svr_lin = grid_svr_lin.best_estimator_

print(f'Das n validacións cruzadas o mellor r2 é {grid_svr_lin.best_score_:.4f},
cuxos parámetros óptimos son {grid_svr_lin.best_params_}')

y_score_hat_train_lin = best_model_svr_lin.predict(x2_train_scaled)
y_score_hat_test_lin = best_model_svr_lin.predict(x2_test_scaled)
print(f'O r2 no mellor modelo linear no conxunto de adestramento é
{r2_score(y_score_train, y_score_hat_train_lin):.4f}')
print(f'O r2 no mellor modelo linear no conxunto de teste é {r2_score(y_score_test,
y_score_hat_test_lin):.4f}\n')
```

```
Das n validacións cruzadas o mellor r2 é 0.8913, cuxos parámetros óptimos son {'C': 1}
O r2 no mellor modelo linear no conxunto de adestramento é 0.8924
O r2 no mellor modelo linear no conxunto de teste é 0.8889
```

```
fig, axs = plt.subplots(1,3, figsize=(16, 5))

residuos = y_score_test - y_score_hat_test_lin
if residuos.mean() < 0.1:
    print(f'O valor medio do residuo no modelo linear é {residuos.mean():.4f}, que é
un valor moi cercano a 0. Danos confianza no axuste.')
else:
    print(f'O valor medio do residuo no modelo linear é {residuos.mean():.4f}, que é
un valor moi grande e sinala unha patoloxía na análise.')

# 1. Gráfico de residuos
sns.scatterplot(x = y_score_hat_test_lin,y = residuos, alpha=0.8, ax=axs[0])
axs[0].axhline(0, color='red', linestyle='--')
axs[0].set_xlabel("Valores preditos")
axs[0].set_ylabel("Residuos")
axs[0].set_title("Análise dos residuos en Linear")
axs[0].grid(alpha=0.6)

# 2. Histograma dos residuos
sns.histplot(residuos, kde=True, bins=10, ax=axs[1])
axs[1].axvline(0, color='red', linestyle='--')
axs[1].set_xlabel("Erro de predición")
axs[1].set_ylabel("Frecuencia")
axs[1].set_title("Distribución dos residuos en Linear")

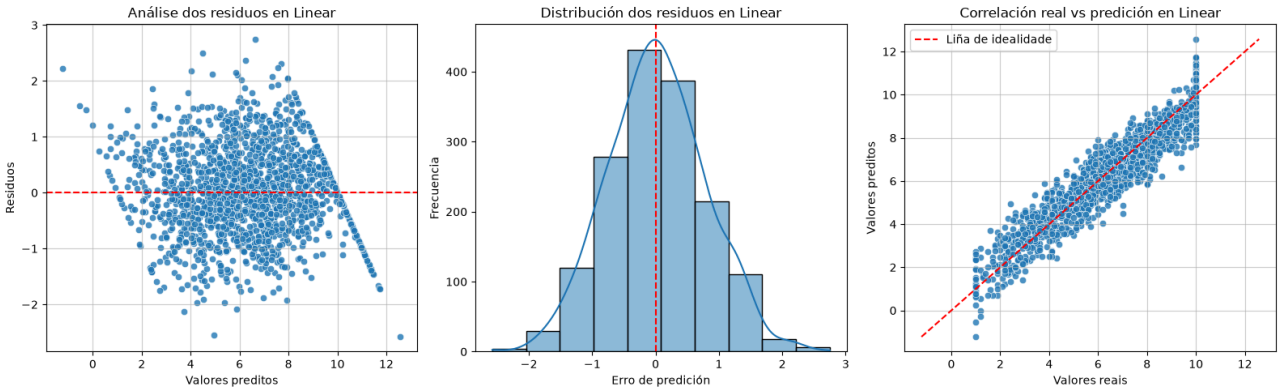
# 3. Valores reais vs preditos
sns.scatterplot(x = y_score_test, y = y_score_hat_test_lin, alpha = 0.8, ax = axs[2])

limites = [
    min(y_score_test.min(), y_score_hat_test_lin.min()),
    max(y_score_test.max(), y_score_hat_test_lin.max())
]

axs[2].plot(limites, limites, 'r--', label='Liña de idealidade')
axs[2].set_xlabel('Valores reais')
axs[2].set_ylabel('Valores preditos')
axs[2].set_title('Correlación real vs predición en Linear')
axs[2].grid(alpha=0.6)
axs[2].legend()

plt.tight_layout()
plt.show()
```

```
O valor medio do residuo no modelo linear é 0.0228, que é un valor moi cercano a 0.
Danos confianza no axuste.
```



3.3.2 Estimación, training e resultados do modelo SVR polinómico

```
param_svr_poly = {
    'C': [1, 10, 100,],
    'gamma': ['auto','scale'],
    'degree': [2, 3, 5, 10],
    'epsilon': [0.2, 0.1],
    'coef0': [0, 1]
}

grid_svr_poly = GridSearchCV( # 6.30 min de compilado
    estimator = SVR(kernel='poly'),
    param_grid = param_svr_poly,
    cv = cv,
    scoring = 'r2', # ver isto
    n_jobs = -1
)

grid_svr_poly.fit(x2_train_scaled, y_score_train)
best_model_svr_poly = grid_svr_poly.best_estimator_

print(f'Das n validacións cruzadas o mellor r2 é {grid_svr_poly.best_score_:.4f},
cuxos parámetros óptimos son {grid_svr_poly.best_params_}')

y_score_hat_train_poly = best_model_svr_poly.predict(x2_train_scaled)
y_score_hat_test_poly = best_model_svr_poly.predict(x2_test_scaled)
print(f'O r2 no mellor modelo polinómico no conxunto de adestramento é
{r2_score(y_score_train, y_score_hat_train_poly):.4f}')
print(f'O r2 no mellor modelo polinómico no conxunto de teste é
{r2_score(y_score_test, y_score_hat_test_poly):.4f}\n')
```

```
Das n validacións cruzadas o mellor r2 é 0.9103, cuxos parámetros óptimos son {'C': 1,
'coef0': 1, 'degree': 3, 'epsilon': 0.2, 'gamma': 'scale'}
O r2 no mellor modelo polinómico no conxunto de adestramento é 0.9336
O r2 no mellor modelo polinómico no conxunto de teste é 0.9173
```

```
fig, axs = plt.subplots(1,3, figsize=(16, 5))

residuos = y_score_test - y_score_hat_test_poly
if residuos.mean() < 0.1:
    print(f'O valor medio do residuo no modelo polinómico é {residuos.mean():.4f},
que é un valor moi cercano a 0. Danos confianza no axuste.')
else:
    print(f'O valor medio do residuo no modelo polinómico é {residuos.mean():.4f},
que é un valor moi grande e sinala unha patoloxía na análise.')

# 1. Gráfico de residuos
sns.scatterplot(x = y_score_hat_test_poly, y = residuos, alpha=0.8, ax=axs[0])
axs[0].axhline(0, color='red', linestyle='--')
axs[0].set_xlabel("Valores preditos")
axs[0].set_ylabel("Residuos")
axs[0].set_title("Análise dos residuos en polinómica")
axs[0].grid(alpha=0.6)

# 2. Histograma dos residuos
sns.histplot(residuos, kde=True, bins=10, ax=axs[1])
axs[1].axvline(0, color='red', linestyle='--')
axs[1].set_xlabel("Erro de predición")
axs[1].set_ylabel("Frecuencia")
axs[1].set_title("Distribución dos residuos en polinómica")

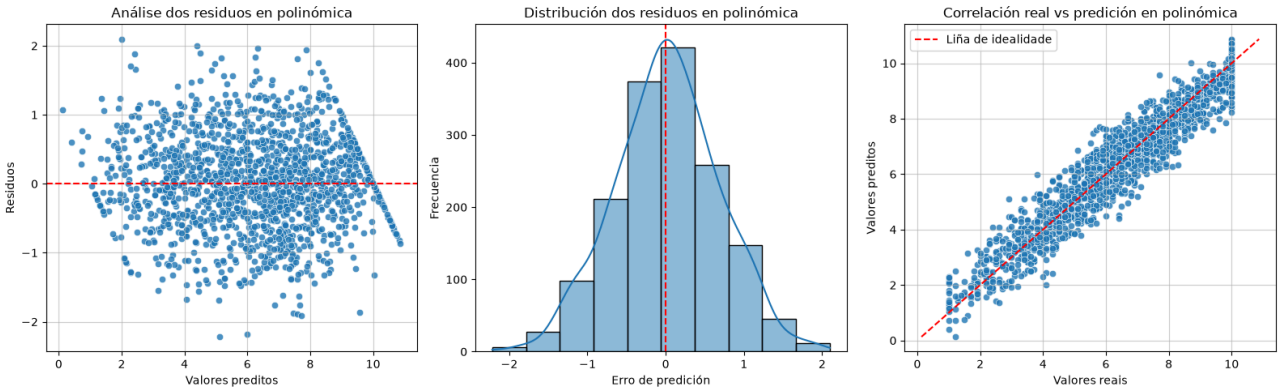
# 3. Valores reais vs preditos
sns.scatterplot(x = y_score_test, y = y_score_hat_test_poly, alpha=0.8, ax = axs[2])

limites = [
    min(y_score_test.min(), y_score_hat_test_poly.min()),
    max(y_score_test.max(), y_score_hat_test_poly.max())
]

axs[2].plot(limites, limites, 'r--', label='Liña de idealidade')
axs[2].set_xlabel('Valores reais')
axs[2].set_ylabel('Valores preditos')
axs[2].set_title('Correlación real vs predición en polinómica')
axs[2].grid(alpha=0.6)
axs[2].legend()

plt.tight_layout()
plt.show()
```

```
O valor medio do residuo no modelo polinómico é 0.0206, que é un valor moi cercano a
0. Danos confianza no axuste.
```



3.3.3 Estimación, training e resultados do modelo SVR RBF

```
param_svr_rbf = {
    'C': [1, 10, 100],
    'gamma': ['auto','scale'],
    'epsilon': [0.1, 0.2]
}

cv = KFold(n_splits=5, shuffle=True, random_state=100) # podemos esixirlle máis
iteracións porque non ten case parámetros que optimizar ~2 min

grid_svr_rbf = GridSearchCV(
    estimator = SVR(kernel='rbf'),
    param_grid = param_svr_rbf,
    cv = cv,
    scoring = 'r2',
    n_jobs = -1
)

grid_svr_rbf.fit(x2_train_scaled, y_score_train)
best_model_svrrbf = grid_svr_rbf.best_estimator_


print(f'Das n validacións cruzadas o mellor r2 é {grid_svr_rbf.best_score_:.4f},
cuxos parámetros óptimos son {grid_svr_rbf.best_params_}')

y_score_hat_train_rbf = best_model_svrrbf.predict(x2_train_scaled)
y_score_hat_test_rbf = best_model_svrrbf.predict(x2_test_scaled)
print(f'O r2 no mellor modelo RBF no conxunto de adestramento é
{r2_score(y_score_train, y_score_hat_train_rbf):.4f}')
print(f'O r2 no mellor modelo RBF no conxunto de teste é {r2_score(y_score_test,
y_score_hat_test_rbf):.4f}\n')
```

```
Das n validacións cruzadas o mellor r2 é 0.9100, cuxos parámetros óptimos son {'C': 1,
'epsilon': 0.2, 'gamma': 'auto'}
O r2 no mellor modelo RBF no conxunto de adestramento é 0.9306
O r2 no mellor modelo RBF no conxunto de teste é 0.9147
```

```
fig, axs = plt.subplots(1,3, figsize=(16, 5))

residuos = y_score_test - y_score_hat_test_rbf
if residuos.mean() < 0.1:
    print(f'O valor medio do residuo no modelo RBF é {residuos.mean():.4f}, que é un
valor moi cercano a 0. Danos confianza no axuste.')
else:
    print(f'O valor medio do residuo no modelo RBF é {residuos.mean():.4f}, que é un
valor moi grande e sinala unha patoloxía na análise.')

# 1. Gráfico de residuos
sns.scatterplot(x = y_score_hat_test_rbf, y = residuos, alpha=0.8, ax=axs[0])
axs[0].axhline(0, color='red', linestyle='--')
axs[0].set_xlabel("Valores preditos")
axs[0].set_ylabel("Residuos")
axs[0].set_title("Análise dos residuos en RBF")
axs[0].grid(alpha=0.5)

# 2. Histograma dos residuos
sns.histplot(residuos, kde=True, bins=10, ax=axs[1])
axs[1].axvline(0, color='red', linestyle='--')
axs[1].set_xlabel("Erro de predición")
axs[1].set_ylabel("Frecuencia")
axs[1].set_title("Distribución dos residuos en RBF")

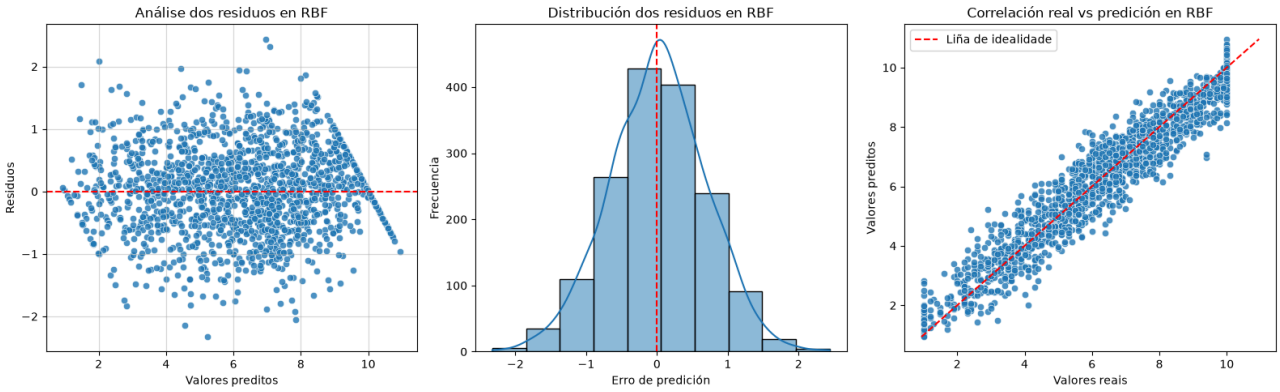
# 3. Valores reais vs preditos
sns.scatterplot(x = y_score_test, y = y_score_hat_test_rbf, alpha = 0.8, ax = axs[2])

limites = [
    min(y_score_test.min(), y_score_hat_test_rbf.min()),
    max(y_score_test.max(), y_score_hat_test_rbf.max())
]

axs[2].plot(limites, limites, 'r--', label='Liña de idealidade')
axs[2].set_xlabel('Valores reais')
axs[2].set_ylabel('Valores preditos')
axs[2].set_title('Correlación real vs predición en RBF')
axs[2].legend()

plt.tight_layout()
plt.show()
```

```
O valor medio do residuo no modelo RBF é 0.0151, que é un valor moi cercano a 0. Danos
confianza no axuste.
```



3.3.4 Conclusións das regresións SVR

Representamos na seguinte táboa os resultados principais das regresións con SVR

Modelo	r2 Modelo	r2 train	r2 teste	media residuos
Linear	0.8913	0.8924	0.8889	0.0228
Poly (n=5)	0.9103	0.9336	0.9173	0.0206
RBF	0.9041	0.9306	0.9147	0.0151

Pódese concluír que o modelo linear, como cabe esperar, é máis limitado que os demais, pero igualmente ten unha moi alta capacidade predictiva sobre a variable "crash_severity_score", de 89 puntos. Aínda así non hai diferenzas moi significativas, o modelo polinómico e rbf están 3 puntos por enriba do lineal, en canto ao polinómico axusta moi lixeiramente mellor que o rbf, a penas 0.3 puntos de diferenza. Todos os modelos minimizan moi ben os residuos, aínda que o rbf faino notablemente mellor que o polinómico. Á hora de falar de custe computacional, buscar o mellor modelo polinómico ten unha carga moi alta (7 min empregando un procesador relativamente potente), fronte ao modelo rbf que axusta case igual (e minimiza mellor o residuo) atopamos que supón un prezo moito menor (tan só 15 segundos). Entre todo, xunto cos resultados anteriores para a columna "crash_risk_level", o modelo rbf supón melloras considerables en case todos os aspectos fronte outros modelos.

1.4 4. Análise con redes neuronais

Nesta sección probamos de novo a predicir os valores de "crash_risk_level" co conxunto de datos sen sesgo, pero esta vez probamos a modelizar unha rede neuronal. Empregamos o paquete keras co lanzador de tensorflow e creamos varios modelos basados en Sequential, conexións neuronais lineares, e como o noso análise é basicamente numérico só temos que usar capas tipo Dense() (para texto, imaxes, audio, etc débense empregar redes convulacionais ou) e capas tipo Dropout() para evitar o overfitting. Empezamos por un modelo moi básico e o melloramos engadindo unha capa de profundida e usando funcións de activación máis suaves para reducir o ruído no loss e ademais impoñemos unha penalización aos pesos grandes (L2) nestas funcións para

evitar que aparezan diverxencias. Este modelo xa funcionaría moi ben, pero imos un paso máis alá; empregamos `keras_tuner` que é un análogo á validación cruzada para neuronas e probamos distintas funcións de activación con distintos parámetros (learning_rate, L2, batch_size, número de neuronas) e con distintos optimizadores (en xeral podemos descartar todos aquelos que integren momento (Adam, Nadam, AdamW), incluso con nesterov, porque aparecen diverxencias moi significativas no loss e os de entreno e validación independínzanse), deixámolo calcular por moitas horas e aínda que os resultados non son nada concluíntes atopamos unha certa tendencia cara cales funcións e parámetros parecen mellores.

Consultar `memoria_traballo.pdf` para máis información sobre a elección dos modelos.

O primeiro que hai que facer é transformar os nosos y'es de str a vectores numéricos dalgunha forma, sklearn ten unha boa opción para facer esta transformación de xeito que as redes neuronais poidan traballar e non perdamos información. Unha consecuencia disto é que debemos ter 4 parámetros categoriales de saída axustados pola rede--> a capa de saída debe ter 4 units e operar coa función softmax.

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

y2_train_enc = le.fit_transform(y2_train)
y2_test_enc = le.transform(y2_test)
```

1.4.4 4.1 Definición dos modelos de redes

Definimos os tres devanditos modelos

```
import tensorflow as tf
tf.keras.utils.set_random_seed(100)

from keras import layers
from keras.models import Sequential
from keras.regularizers import L1L2, L2, L1
from keras.optimizers import SGD, Adam, Nadam, AdamW
from keras.optimizers.schedules import ExponentialDecay

# initial_ler = 0.02
# lr_exp = ExponentialDecay(
#     initial_ler, decay_steps=1000, decay_rate=0.9, staircase=True
# )

def modelo_simple(data): # x2_train_scaled

    model = Sequential([
        layers.Input(shape=(data.shape[1],)),
```

```
        layers.Dense(32, activation='relu'),
        layers.Dropout(0.1),
        layers.Dense(16, activation='relu'),
        layers.Dense(4, activation='softmax')
        ])

    model.compile(optimizer = SGD(learning_rate = 0.0009
                ), loss = 'sparse_categorical_crossentropy', metrics = ['accuracy'])
# 90 iteracións son suficientes

    return model

def modelo_mellorado(data): # x2_train_scaled

    model = Sequential([
        layers.Input(shape=(data.shape[1],)),
        layers.Dense(64, activation='celu', kernel_initializer='he_normal',
kernel_regularizer= L2(0.01)),
        layers.Dropout(0.1),
        layers.Dense(32, activation='celu', kernel_initializer='he_normal',
kernel_regularizer= L2(0.01)),
        layers.Dense(16, activation='celu', kernel_initializer='he_normal',
kernel_regularizer= L2(0.01)),
        layers.Dense(4, activation='softmax')
    ])

    model.compile(optimizer = SGD(learning_rate = 0.001
                ), loss = 'sparse_categorical_crossentropy', metrics = ['accuracy'])

    return model

def modelo_optimizado(data): # Aínda que o modelo pareza considerablemente máis
custoso computacionalmente que os anteriores, este permitiranos conseguir
    # resultados moi semellantes a modelo_mellorado con moitas menos iteracións
(epochs = 60 ou 70) durante o entrenamento. Se poñemos moitas iteracións no modelo
    # mellorado podemos decatarnos que entre as 50 e 150 iteracións hai un val de
precisión e temos que superar ese límite de 150 para predicións estables. No entanto,
    # este modelo optimizado permite chegar un máximo de predición semellante que é
basicamente constante despois das ~75 iteracións reducindo así moitimo o tempo de
compilación

    model = Sequential([
        layers.Input(shape=(data.shape[1],)),
        layers.Dense(128, activation='silu', kernel_initializer='glorot_uniform' ,
kernel_regularizer= L1L2(l1 = 0.0002, l2 = 0.01)), # silu parece mellorar un pelin a
puntuación
        layers.Dropout(0.1),
        layers.Dense(16, activation='celu', kernel_initializer='he_normal',
kernel_regularizer= L2(0.01)),
        layers.Dense(40, activation='celu', kernel_initializer='he_normal',
kernel_regularizer= L2(0.01)),
        layers.Dropout(0.1),
        layers.Dense(16, activation='relu', kernel_initializer='he_normal',
kernel_regularizer= L2(0.01)), # probo entre relu e celu
```

```
        layers.Dense(4, activation='softmax')
        ])

    model.compile(optimizer = SGD(learning_rate = 0.001
                ), loss = 'sparse_categorical_crossentropy', metrics = ['accuracy'])

    return model


modelo_simple(x2_train_scaled).summary()
modelo_mellorado(x2_train_scaled).summary()
modelo_optimizado(x2_train_scaled).summary()
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow ⩾ 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use
WSL2 or the TensorFlow-DirectML plugin.
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32)	736
dropout (Dropout)	(None, 32)	0
dense_1 (Dense)	(None, 16)	528
dense_2 (Dense)	(None, 4)	68

```
Total params: 1,332 (5.20 KB)

Trainable params: 1,332 (5.20 KB)

Non-trainable params: 0 (0.00 B)
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 64)	1,472
dropout_1 (Dropout)	(None, 64)	0
dense_4 (Dense)	(None, 32)	2,080

```
┃ dense_5 (Dense)                 ┃ (None, 16)             ┃           528 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━┫
┃ dense_6 (Dense)                 ┃ (None, 4)              ┃            68 ┃
┗━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━━━━┛
 Total params: 4,148 (16.20 KB)
 Trainable params: 4,148 (16.20 KB)
 Non-trainable params: 0 (0.00 B)
Model: "sequential_2"
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape           ┃       Param # ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━┫
┃ dense_7 (Dense)                 ┃ (None, 128)            ┃         2,944 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━┫
┃ dropout_2 (Dropout)             ┃ (None, 128)            ┃             0 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━┫
┃ dense_8 (Dense)                 ┃ (None, 16)             ┃         2,064 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━┫
┃ dense_9 (Dense)                 ┃ (None, 40)             ┃           680 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━┫
┃ dropout_3 (Dropout)             ┃ (None, 40)             ┃             0 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━┫
┃ dense_10 (Dense)                ┃ (None, 16)             ┃           656 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━┫
┃ dense_11 (Dense)                ┃ (None, 4)              ┃            68 ┃
┗━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━━━━┛
 Total params: 6,412 (25.05 KB)
 Trainable params: 6,412 (25.05 KB)
 Non-trainable params: 0 (0.00 B)
```

1.4.5 4.2 Entrenamento das redes neuronais

Toca entrenar as nosas redes neuronais, como entrenar 3 redes á vez leva un tempo bastante considerable podemos simplemente comentar aquelas que non nos interesen, deixamos só o entrenamento do modelo optimizado que será da que consideraremos resultados perdictivos

```
modelo1 = modelo_simple(x2_train_scaled)
cerebro_simple = modelo1.fit(
    x2_train_scaled,
    y2_train_enc,
    epochs=70,
    batch_size=8,
    validation_data = (x2_test_scaled,y2_test_enc)
)

modelo2 = modelo_mellorado(x2_train_scaled)
cerebro_mellorado = modelo2.fit(
    x2_train_scaled,
    y2_train_enc,
    epochs=70,
    batch_size=8,
    validation_data = (x2_test_scaled,y2_test_enc)
)

modelo3 = modelo_optimizado(x2_train_scaled)
cerebro_optimizado = modelo3.fit(
    x2_train_scaled,
    y2_train_enc,
    epochs=70,
    batch_size=8,
    validation_data = (x2_test_scaled,y2_test_enc)
)

# e se probo a facer unha nova subdivisión con validation_split? Realmente non sei o
que sería correcto entre as dúas opcións.
```

```
Epoch 1/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.3136 - loss: 1.4088 -
val_accuracy: 0.3994 - val_loss: 1.2738
Epoch 2/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.4222 - loss: 1.2401 -
val_accuracy: 0.5106 - val_loss: 1.1591
Epoch 3/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.4814 - loss: 1.1441 -
val_accuracy: 0.5663 - val_loss: 1.0731
Epoch 4/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.5316 - loss: 1.0655 -
val_accuracy: 0.5838 - val_loss: 1.0018
Epoch 5/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.5512 - loss: 1.0048 -
val_accuracy: 0.6025 - val_loss: 0.9437
Epoch 6/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.5780 - loss: 0.9489 -
val_accuracy: 0.6194 - val_loss: 0.8950
Epoch 7/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.5992 - loss: 0.9060 -
val_accuracy: 0.6438 - val_loss: 0.8551
Epoch 8/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.6191 - loss: 0.8682 -
```

```
val_accuracy: 0.6619 - val_loss: 0.8201
Epoch 9/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.6300 - loss: 0.8338 -
val_accuracy: 0.6750 - val_loss: 0.7892
Epoch 10/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.6463 - loss: 0.8027 -
val_accuracy: 0.6931 - val_loss: 0.7609
Epoch 11/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.6562 - loss: 0.7806 -
val_accuracy: 0.7006 - val_loss: 0.7363
Epoch 12/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.6664 - loss: 0.7571 -
val_accuracy: 0.7100 - val_loss: 0.7149
Epoch 13/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.6745 - loss: 0.7415 -
val_accuracy: 0.7188 - val_loss: 0.6952
Epoch 14/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.6914 - loss: 0.7177 -
val_accuracy: 0.7281 - val_loss: 0.6777
Epoch 15/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.6922 - loss: 0.7028 -
val_accuracy: 0.7387 - val_loss: 0.6615
Epoch 16/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.6936 - loss: 0.6914 -
val_accuracy: 0.7381 - val_loss: 0.6478
Epoch 17/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7013 - loss: 0.6743 -
val_accuracy: 0.7450 - val_loss: 0.6348
Epoch 18/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7116 - loss: 0.6612 -
val_accuracy: 0.7487 - val_loss: 0.6231
Epoch 19/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.7084 - loss: 0.6560 -
val_accuracy: 0.7500 - val_loss: 0.6124
Epoch 20/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.7144 - loss: 0.6424 -
val_accuracy: 0.7538 - val_loss: 0.6034
Epoch 21/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7172 - loss: 0.6385 -
val_accuracy: 0.7569 - val_loss: 0.5948
Epoch 22/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7138 - loss: 0.6317 -
val_accuracy: 0.7600 - val_loss: 0.5872
Epoch 23/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7256 - loss: 0.6161 -
val_accuracy: 0.7663 - val_loss: 0.5797
Epoch 24/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7330 - loss: 0.6063 -
val_accuracy: 0.7675 - val_loss: 0.5727
Epoch 25/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7384 - loss: 0.5976 -
val_accuracy: 0.7725 - val_loss: 0.5667
Epoch 26/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7317 - loss: 0.6001 -
val_accuracy: 0.7756 - val_loss: 0.5609
```

```
Epoch 27/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.7384 - loss: 0.5897 -
val_accuracy: 0.7725 - val_loss: 0.5556
Epoch 28/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7333 - loss: 0.5945 -
val_accuracy: 0.7713 - val_loss: 0.5510
Epoch 29/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7391 - loss: 0.5853 -
val_accuracy: 0.7700 - val_loss: 0.5469
Epoch 30/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7420 - loss: 0.5739 -
val_accuracy: 0.7706 - val_loss: 0.5425
Epoch 31/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.7391 - loss: 0.5727 -
val_accuracy: 0.7731 - val_loss: 0.5392
Epoch 32/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.7517 - loss: 0.5688 -
val_accuracy: 0.7719 - val_loss: 0.5358
Epoch 33/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7466 - loss: 0.5692 -
val_accuracy: 0.7719 - val_loss: 0.5332
Epoch 34/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.7425 - loss: 0.5694 -
val_accuracy: 0.7750 - val_loss: 0.5300
Epoch 35/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7494 - loss: 0.5597 -
val_accuracy: 0.7750 - val_loss: 0.5276
Epoch 36/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7575 - loss: 0.5552 -
val_accuracy: 0.7775 - val_loss: 0.5251
Epoch 37/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7541 - loss: 0.5551 -
val_accuracy: 0.7775 - val_loss: 0.5224
Epoch 38/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7623 - loss: 0.5450 -
val_accuracy: 0.7800 - val_loss: 0.5201
Epoch 39/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7534 - loss: 0.5492 -
val_accuracy: 0.7788 - val_loss: 0.5178
Epoch 40/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7575 - loss: 0.5474 -
val_accuracy: 0.7775 - val_loss: 0.5159
Epoch 41/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7606 - loss: 0.5414 -
val_accuracy: 0.7788 - val_loss: 0.5134
Epoch 42/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7622 - loss: 0.5343 -
val_accuracy: 0.7781 - val_loss: 0.5112
Epoch 43/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7603 - loss: 0.5364 -
val_accuracy: 0.7781 - val_loss: 0.5103
Epoch 44/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7616 - loss: 0.5327 -
val_accuracy: 0.7806 - val_loss: 0.5079
Epoch 45/70
```

```
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.7608 - loss: 0.5364 -
val_accuracy: 0.7788 - val_loss: 0.5065
Epoch 46/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.7658 - loss: 0.5284 -
val_accuracy: 0.7794 - val_loss: 0.5054
Epoch 47/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7652 - loss: 0.5306 -
val_accuracy: 0.7831 - val_loss: 0.5037
Epoch 48/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7686 - loss: 0.5288 -
val_accuracy: 0.7819 - val_loss: 0.5020
Epoch 49/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7609 - loss: 0.5289 -
val_accuracy: 0.7837 - val_loss: 0.5011
Epoch 50/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7686 - loss: 0.5243 -
val_accuracy: 0.7862 - val_loss: 0.4996
Epoch 51/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.7661 - loss: 0.5256 -
val_accuracy: 0.7875 - val_loss: 0.4989
Epoch 52/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7739 - loss: 0.5182 -
val_accuracy: 0.7875 - val_loss: 0.4975
Epoch 53/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7697 - loss: 0.5229 -
val_accuracy: 0.7894 - val_loss: 0.4961
Epoch 54/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7733 - loss: 0.5208 -
val_accuracy: 0.7881 - val_loss: 0.4953
Epoch 55/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7728 - loss: 0.5214 -
val_accuracy: 0.7881 - val_loss: 0.4940
Epoch 56/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7773 - loss: 0.5077 -
val_accuracy: 0.7856 - val_loss: 0.4931
Epoch 57/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7702 - loss: 0.5158 -
val_accuracy: 0.7881 - val_loss: 0.4919
Epoch 58/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.7722 - loss: 0.5145 -
val_accuracy: 0.7887 - val_loss: 0.4908
Epoch 59/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7731 - loss: 0.5106 -
val_accuracy: 0.7912 - val_loss: 0.4903
Epoch 60/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7719 - loss: 0.5137 -
val_accuracy: 0.7912 - val_loss: 0.4886
Epoch 61/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7753 - loss: 0.5103 -
val_accuracy: 0.7894 - val_loss: 0.4876
Epoch 62/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7714 - loss: 0.5150 -
val_accuracy: 0.7912 - val_loss: 0.4870
Epoch 63/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7745 - loss: 0.5047 -
```

```
val_accuracy: 0.7944 - val_loss: 0.4861
Epoch 64/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7744 - loss: 0.5091 -
val_accuracy: 0.7944 - val_loss: 0.4854
Epoch 65/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7753 - loss: 0.5070 -
val_accuracy: 0.7950 - val_loss: 0.4848
Epoch 66/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7758 - loss: 0.5013 -
val_accuracy: 0.7969 - val_loss: 0.4845
Epoch 67/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7736 - loss: 0.5059 -
val_accuracy: 0.7950 - val_loss: 0.4836
Epoch 68/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7772 - loss: 0.5027 -
val_accuracy: 0.7937 - val_loss: 0.4826
Epoch 69/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7759 - loss: 0.5101 -
val_accuracy: 0.7962 - val_loss: 0.4821
Epoch 70/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7805 - loss: 0.4995 -
val_accuracy: 0.7962 - val_loss: 0.4819
Epoch 1/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.3802 - loss: 3.5829 -
val_accuracy: 0.4863 - val_loss: 3.2451
Epoch 2/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.5286 - loss: 3.1589 -
val_accuracy: 0.5800 - val_loss: 3.0202
Epoch 3/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.5850 - loss: 2.9774 -
val_accuracy: 0.6300 - val_loss: 2.8716
Epoch 4/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.6261 - loss: 2.8468 -
val_accuracy: 0.6525 - val_loss: 2.7536
Epoch 5/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.6539 - loss: 2.7321 -
val_accuracy: 0.6787 - val_loss: 2.6533
Epoch 6/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.6720 - loss: 2.6428 -
val_accuracy: 0.6906 - val_loss: 2.5645
Epoch 7/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.6892 - loss: 2.5561 -
val_accuracy: 0.7100 - val_loss: 2.4837
Epoch 8/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.6884 - loss: 2.4823 -
val_accuracy: 0.7200 - val_loss: 2.4091
Epoch 9/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7130 - loss: 2.4043 -
val_accuracy: 0.7250 - val_loss: 2.3399
Epoch 10/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7172 - loss: 2.3389 -
val_accuracy: 0.7281 - val_loss: 2.2762
Epoch 11/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7217 - loss: 2.2735 -
val_accuracy: 0.7362 - val_loss: 2.2156
```

```
Epoch 12/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7192 - loss: 2.2178 -
val_accuracy: 0.7387 - val_loss: 2.1570
Epoch 13/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7330 - loss: 2.1572 -
val_accuracy: 0.7437 - val_loss: 2.1021
Epoch 14/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7325 - loss: 2.1029 -
val_accuracy: 0.7469 - val_loss: 2.0504
Epoch 15/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7359 - loss: 2.0529 -
val_accuracy: 0.7494 - val_loss: 2.0000
Epoch 16/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7467 - loss: 1.9961 -
val_accuracy: 0.7544 - val_loss: 1.9527
Epoch 17/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7473 - loss: 1.9570 -
val_accuracy: 0.7531 - val_loss: 1.9055
Epoch 18/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7498 - loss: 1.9078 -
val_accuracy: 0.7600 - val_loss: 1.8624
Epoch 19/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7520 - loss: 1.8648 -
val_accuracy: 0.7625 - val_loss: 1.8193
Epoch 20/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7500 - loss: 1.8242 -
val_accuracy: 0.7650 - val_loss: 1.7782
Epoch 21/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7584 - loss: 1.7757 -
val_accuracy: 0.7644 - val_loss: 1.7388
Epoch 22/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7591 - loss: 1.7373 -
val_accuracy: 0.7688 - val_loss: 1.7002
Epoch 23/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7548 - loss: 1.7030 -
val_accuracy: 0.7700 - val_loss: 1.6638
Epoch 24/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7592 - loss: 1.6649 -
val_accuracy: 0.7706 - val_loss: 1.6285
Epoch 25/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7591 - loss: 1.6287 -
val_accuracy: 0.7700 - val_loss: 1.5945
Epoch 26/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7653 - loss: 1.5967 -
val_accuracy: 0.7750 - val_loss: 1.5619
Epoch 27/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7659 - loss: 1.5603 -
val_accuracy: 0.7750 - val_loss: 1.5295
Epoch 28/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7680 - loss: 1.5300 -
val_accuracy: 0.7775 - val_loss: 1.4986
Epoch 29/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7627 - loss: 1.5024 -
val_accuracy: 0.7731 - val_loss: 1.4689
Epoch 30/70
```

```
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7678 - loss: 1.4691 -
val_accuracy: 0.7763 - val_loss: 1.4407
Epoch 31/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7689 - loss: 1.4371 -
val_accuracy: 0.7756 - val_loss: 1.4118
Epoch 32/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7723 - loss: 1.4078 -
val_accuracy: 0.7781 - val_loss: 1.3852
Epoch 33/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7739 - loss: 1.3803 -
val_accuracy: 0.7800 - val_loss: 1.3594
Epoch 34/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7755 - loss: 1.3547 -
val_accuracy: 0.7794 - val_loss: 1.3346
Epoch 35/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7663 - loss: 1.3346 -
val_accuracy: 0.7806 - val_loss: 1.3099
Epoch 36/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7716 - loss: 1.3080 -
val_accuracy: 0.7806 - val_loss: 1.2864
Epoch 37/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7761 - loss: 1.2862 -
val_accuracy: 0.7806 - val_loss: 1.2638
Epoch 38/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7741 - loss: 1.2651 -
val_accuracy: 0.7794 - val_loss: 1.2414
Epoch 39/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7786 - loss: 1.2413 -
val_accuracy: 0.7844 - val_loss: 1.2203
Epoch 40/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7739 - loss: 1.2187 -
val_accuracy: 0.7825 - val_loss: 1.2001
Epoch 41/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7739 - loss: 1.1991 -
val_accuracy: 0.7800 - val_loss: 1.1799
Epoch 42/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7795 - loss: 1.1753 -
val_accuracy: 0.7837 - val_loss: 1.1602
Epoch 43/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7775 - loss: 1.1576 -
val_accuracy: 0.7837 - val_loss: 1.1419
Epoch 44/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7841 - loss: 1.1335 -
val_accuracy: 0.7831 - val_loss: 1.1243
Epoch 45/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7731 - loss: 1.1223 -
val_accuracy: 0.7812 - val_loss: 1.1060
Epoch 46/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7830 - loss: 1.1015 -
val_accuracy: 0.7819 - val_loss: 1.0890
Epoch 47/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7789 - loss: 1.0870 -
val_accuracy: 0.7825 - val_loss: 1.0740
Epoch 48/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7759 - loss: 1.0711 -
```

```
val_accuracy: 0.7869 - val_loss: 1.0572
Epoch 49/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7869 - loss: 1.0525 -
val_accuracy: 0.7850 - val_loss: 1.0422
Epoch 50/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7841 - loss: 1.0378 -
val_accuracy: 0.7856 - val_loss: 1.0273
Epoch 51/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7814 - loss: 1.0257 -
val_accuracy: 0.7837 - val_loss: 1.0138
Epoch 52/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7855 - loss: 1.0072 -
val_accuracy: 0.7831 - val_loss: 0.9996
Epoch 53/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7861 - loss: 0.9939 -
val_accuracy: 0.7856 - val_loss: 0.9863
Epoch 54/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7816 - loss: 0.9852 -
val_accuracy: 0.7831 - val_loss: 0.9728
Epoch 55/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7827 - loss: 0.9750 -
val_accuracy: 0.7837 - val_loss: 0.9609
Epoch 56/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7866 - loss: 0.9553 -
val_accuracy: 0.7812 - val_loss: 0.9483
Epoch 57/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7836 - loss: 0.9445 -
val_accuracy: 0.7844 - val_loss: 0.9362
Epoch 58/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7822 - loss: 0.9324 -
val_accuracy: 0.7806 - val_loss: 0.9250
Epoch 59/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7884 - loss: 0.9191 -
val_accuracy: 0.7837 - val_loss: 0.9134
Epoch 60/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7841 - loss: 0.9110 -
val_accuracy: 0.7844 - val_loss: 0.9031
Epoch 61/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7881 - loss: 0.8994 -
val_accuracy: 0.7819 - val_loss: 0.8933
Epoch 62/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7889 - loss: 0.8896 -
val_accuracy: 0.7819 - val_loss: 0.8826
Epoch 63/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7856 - loss: 0.8758 -
val_accuracy: 0.7831 - val_loss: 0.8725
Epoch 64/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7839 - loss: 0.8699 -
val_accuracy: 0.7806 - val_loss: 0.8631
Epoch 65/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7914 - loss: 0.8577 -
val_accuracy: 0.7850 - val_loss: 0.8541
Epoch 66/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7861 - loss: 0.8500 -
val_accuracy: 0.7850 - val_loss: 0.8457
```

```
Epoch 67/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7869 - loss: 0.8388 -
val_accuracy: 0.7806 - val_loss: 0.8371
Epoch 68/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7886 - loss: 0.8336 -
val_accuracy: 0.7825 - val_loss: 0.8289
Epoch 69/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7900 - loss: 0.8222 -
val_accuracy: 0.7837 - val_loss: 0.8212
Epoch 70/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7912 - loss: 0.8114 -
val_accuracy: 0.7862 - val_loss: 0.8127
Epoch 1/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.3905 - loss: 3.1061 -
val_accuracy: 0.4450 - val_loss: 2.9762
Epoch 2/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.5009 - loss: 2.8826 -
val_accuracy: 0.5431 - val_loss: 2.7924
Epoch 3/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.5736 - loss: 2.7073 -
val_accuracy: 0.5994 - val_loss: 2.6279
Epoch 4/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.6189 - loss: 2.5646 -
val_accuracy: 0.6344 - val_loss: 2.4882
Epoch 5/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.6519 - loss: 2.4392 -
val_accuracy: 0.6694 - val_loss: 2.3709
Epoch 6/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.6778 - loss: 2.3357 -
val_accuracy: 0.6925 - val_loss: 2.2737
Epoch 7/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7044 - loss: 2.2448 -
val_accuracy: 0.7175 - val_loss: 2.1867
Epoch 8/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7198 - loss: 2.1639 -
val_accuracy: 0.7344 - val_loss: 2.1124
Epoch 9/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7320 - loss: 2.0906 -
val_accuracy: 0.7531 - val_loss: 2.0439
Epoch 10/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7427 - loss: 2.0271 -
val_accuracy: 0.7550 - val_loss: 1.9842
Epoch 11/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7442 - loss: 1.9702 -
val_accuracy: 0.7581 - val_loss: 1.9296
Epoch 12/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7498 - loss: 1.9157 -
val_accuracy: 0.7606 - val_loss: 1.8798
Epoch 13/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7542 - loss: 1.8697 -
val_accuracy: 0.7700 - val_loss: 1.8325
Epoch 14/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7592 - loss: 1.8247 -
val_accuracy: 0.7663 - val_loss: 1.7887
Epoch 15/70
```

```
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7650 - loss: 1.7761 -
val_accuracy: 0.7688 - val_loss: 1.7468
Epoch 16/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7628 - loss: 1.7375 -
val_accuracy: 0.7719 - val_loss: 1.7069
Epoch 17/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.7708 - loss: 1.6971 -
val_accuracy: 0.7756 - val_loss: 1.6687
Epoch 18/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7770 - loss: 1.6573 -
val_accuracy: 0.7781 - val_loss: 1.6339
Epoch 19/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7766 - loss: 1.6245 -
val_accuracy: 0.7781 - val_loss: 1.5990
Epoch 20/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7831 - loss: 1.5877 -
val_accuracy: 0.7781 - val_loss: 1.5662
Epoch 21/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7808 - loss: 1.5543 -
val_accuracy: 0.7763 - val_loss: 1.5347
Epoch 22/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7763 - loss: 1.5285 -
val_accuracy: 0.7756 - val_loss: 1.5048
Epoch 23/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7764 - loss: 1.4957 -
val_accuracy: 0.7788 - val_loss: 1.4758
Epoch 24/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7839 - loss: 1.4638 -
val_accuracy: 0.7806 - val_loss: 1.4485
Epoch 25/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7836 - loss: 1.4395 -
val_accuracy: 0.7800 - val_loss: 1.4203
Epoch 26/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7831 - loss: 1.4103 -
val_accuracy: 0.7788 - val_loss: 1.3953
Epoch 27/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7780 - loss: 1.3849 -
val_accuracy: 0.7831 - val_loss: 1.3695
Epoch 28/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7872 - loss: 1.3581 -
val_accuracy: 0.7837 - val_loss: 1.3461
Epoch 29/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7872 - loss: 1.3332 -
val_accuracy: 0.7844 - val_loss: 1.3221
Epoch 30/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7880 - loss: 1.3092 -
val_accuracy: 0.7869 - val_loss: 1.2997
Epoch 31/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7891 - loss: 1.2861 -
val_accuracy: 0.7844 - val_loss: 1.2776
Epoch 32/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7920 - loss: 1.2677 -
val_accuracy: 0.7831 - val_loss: 1.2565
Epoch 33/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7916 - loss: 1.2429 -
```

```
val_accuracy: 0.7831 - val_loss: 1.2362
Epoch 34/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7919 - loss: 1.2240 -
val_accuracy: 0.7856 - val_loss: 1.2160
Epoch 35/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7908 - loss: 1.2051 -
val_accuracy: 0.7856 - val_loss: 1.1972
Epoch 36/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7912 - loss: 1.1850 -
val_accuracy: 0.7850 - val_loss: 1.1786
Epoch 37/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7939 - loss: 1.1665 -
val_accuracy: 0.7856 - val_loss: 1.1602
Epoch 38/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7933 - loss: 1.1462 -
val_accuracy: 0.7850 - val_loss: 1.1431
Epoch 39/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7912 - loss: 1.1321 -
val_accuracy: 0.7837 - val_loss: 1.1260
Epoch 40/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7916 - loss: 1.1150 -
val_accuracy: 0.7831 - val_loss: 1.1092
Epoch 41/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7992 - loss: 1.0933 -
val_accuracy: 0.7837 - val_loss: 1.0934
Epoch 42/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7947 - loss: 1.0821 -
val_accuracy: 0.7837 - val_loss: 1.0781
Epoch 43/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7928 - loss: 1.0687 -
val_accuracy: 0.7881 - val_loss: 1.0634
Epoch 44/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7944 - loss: 1.0554 -
val_accuracy: 0.7862 - val_loss: 1.0497
Epoch 45/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7944 - loss: 1.0380 -
val_accuracy: 0.7875 - val_loss: 1.0356
Epoch 46/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7950 - loss: 1.0234 -
val_accuracy: 0.7862 - val_loss: 1.0217
Epoch 47/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7952 - loss: 1.0105 -
val_accuracy: 0.7862 - val_loss: 1.0087
Epoch 48/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7945 - loss: 0.9983 -
val_accuracy: 0.7869 - val_loss: 0.9962
Epoch 49/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7972 - loss: 0.9882 -
val_accuracy: 0.7881 - val_loss: 0.9843
Epoch 50/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7942 - loss: 0.9755 -
val_accuracy: 0.7875 - val_loss: 0.9722
Epoch 51/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7906 - loss: 0.9659 -
val_accuracy: 0.7881 - val_loss: 0.9597
```

```
Epoch 52/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7927 - loss: 0.9521 -
val_accuracy: 0.7894 - val_loss: 0.9495
Epoch 53/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7944 - loss: 0.9433 -
val_accuracy: 0.7881 - val_loss: 0.9391
Epoch 54/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7970 - loss: 0.9325 -
val_accuracy: 0.7912 - val_loss: 0.9289
Epoch 55/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7981 - loss: 0.9231 -
val_accuracy: 0.7925 - val_loss: 0.9191
Epoch 56/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7937 - loss: 0.9138 -
val_accuracy: 0.7887 - val_loss: 0.9083
Epoch 57/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7977 - loss: 0.9009 -
val_accuracy: 0.7900 - val_loss: 0.9001
Epoch 58/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7962 - loss: 0.8936 -
val_accuracy: 0.7912 - val_loss: 0.8906
Epoch 59/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7972 - loss: 0.8819 -
val_accuracy: 0.7912 - val_loss: 0.8809
Epoch 60/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7944 - loss: 0.8754 -
val_accuracy: 0.7912 - val_loss: 0.8725
Epoch 61/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7997 - loss: 0.8640 -
val_accuracy: 0.7912 - val_loss: 0.8639
Epoch 62/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7942 - loss: 0.8579 -
val_accuracy: 0.7931 - val_loss: 0.8561
Epoch 63/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7977 - loss: 0.8502 -
val_accuracy: 0.7900 - val_loss: 0.8483
Epoch 64/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7950 - loss: 0.8417 -
val_accuracy: 0.7919 - val_loss: 0.8403
Epoch 65/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7923 - loss: 0.8388 -
val_accuracy: 0.7937 - val_loss: 0.8324
Epoch 66/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7983 - loss: 0.8277 -
val_accuracy: 0.7950 - val_loss: 0.8263
Epoch 67/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7975 - loss: 0.8218 -
val_accuracy: 0.7950 - val_loss: 0.8193
Epoch 68/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7986 - loss: 0.8153 -
val_accuracy: 0.7956 - val_loss: 0.8127
Epoch 69/70
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7959 - loss: 0.8074 -
val_accuracy: 0.7937 - val_loss: 0.8060
Epoch 70/70
```

```
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7967 - loss: 0.8041 -
val_accuracy: 0.7956 - val_loss: 0.7995
```

1.4.6 4.3 Resultados

```
# poñer as gráficas e tal
print('o valor final da precisión no entrenamento da rede simple é ',
cerebro_simple.history['accuracy'][-1])
print('o valor final da precisión no teste da rede simple é ',
cerebro_simple.history['val_accuracy'][-1])
print('o valor final do loss no entrenamento da rede simple é ',
cerebro_simple.history['loss'][-1])
print('o valor final do loss no teste da rede simple é ',
cerebro_simple.history['val_loss'][-1], '\n')

print('o valor final da precisión no entrenamento da rede mellorada é ',
cerebro_mellorado.history['accuracy'][-1])
print('o valor final da precisión no teste da rede mellorada é ',
cerebro_mellorado.history['val_accuracy'][-1])
print('o valor final do loss no entrenamento da rede mellorada é ',
cerebro_mellorado.history['loss'][-1])
print('o valor final do loss no teste da rede mellorada é ',
cerebro_mellorado.history['val_loss'][-1], '\n')

print('o valor final da precisión no entrenamento da rede optimizada é ',
cerebro_optimizado.history['accuracy'][-1])
print('o valor final da precisión no teste da rede optimizada é ',
cerebro_optimizado.history['val_accuracy'][-1])
print('o valor final do loss no entrenamento da rede optimizada é ',
cerebro_optimizado.history['loss'][-1])
print('o valor final do loss no teste da rede optimizada é ',
cerebro_optimizado.history['val_loss'][-1], '\n')


fig, axs = plt.subplots(3,2, figsize=(16, 14))

axs[0,0].plot(cerebro_simple.history['loss'], label = 'loss')
axs[0,0].plot(cerebro_simple.history['val_loss'], label =  'val_loss')
axs[0,0].legend(title=f'cerebro simple', loc='lower left')
axs[0,1].plot(cerebro_simple.history['accuracy'], label = 'loss')
axs[0,1].plot(cerebro_simple.history['val_accuracy'], label =  'val_loss')
axs[0,1].legend(title=f'cerebro simple', loc='lower right')

axs[1,0].plot(cerebro_mellorado.history['loss'], label = 'loss')
axs[1,0].plot(cerebro_mellorado.history['val_loss'], label =  'val_loss')
axs[1,0].legend(title=f'cerebro mellorado', loc='lower left')
axs[1,1].plot(cerebro_mellorado.history['accuracy'], label = 'loss')
axs[1,1].plot(cerebro_mellorado.history['val_accuracy'], label =  'val_loss')
axs[1,1].legend(title=f'cerebro mellorado', loc='lower right')

axs[2,0].plot(cerebro_optimizado.history['loss'], label = 'loss')
axs[2,0].plot(cerebro_optimizado.history['val_loss'], label =  'val_loss')
axs[2,0].legend(title=f'cerebro optimizado', loc='lower left')
```

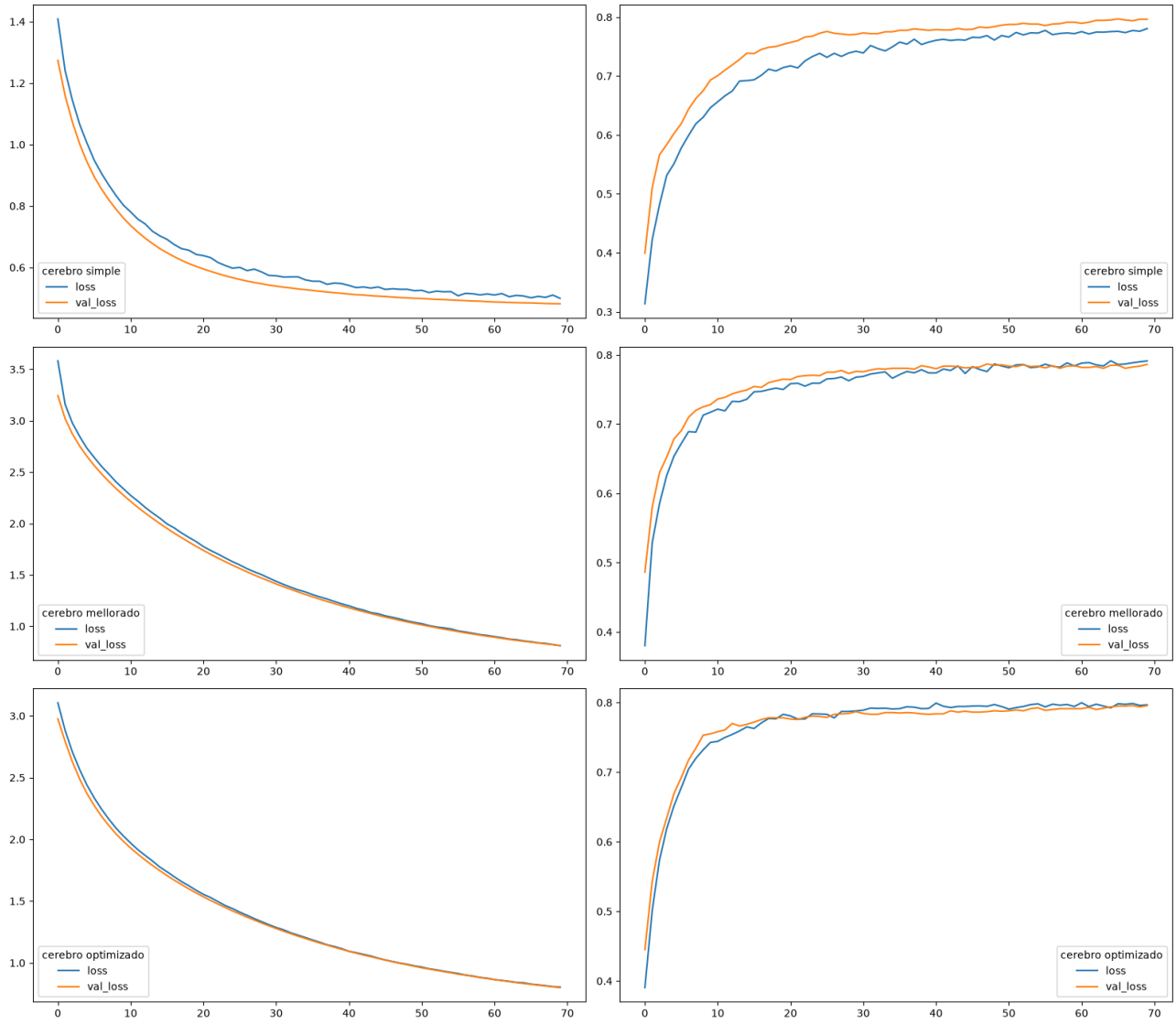
```
axs[2,1].plot(cerebro_optimizado.history['accuracy'], label = 'loss')
axs[2,1].plot(cerebro_optimizado.history['val_accuracy'], label =  'val_loss')
axs[2,1].legend(title=f'cerebro optimizado', loc='lower right')

plt.tight_layout()
fig.show()
```

```
o valor final da precisión no entrenamento da rede simple é  0.780468761920929
o valor final da precisión no teste da rede simple é  0.7962499856948853
o valor final do loss no entrenamento da rede simple é  0.49949055910110474
o valor final do loss no teste da rede simple é  0.4818542003631592

o valor final da precisión no entrenamento da rede mellorada é  0.7912499904632568
o valor final da precisión no teste da rede mellorada é  0.7862499952316284
o valor final do loss no entrenamento da rede mellorada é  0.8114246129989624
o valor final do loss no teste da rede mellorada é  0.8127349615097046

o valor final da precisión no entrenamento da rede optimizada é  0.7967187762260437
o valor final da precisión no teste da rede optimizada é  0.7956249713897705
o valor final do loss no entrenamento da rede optimizada é  0.8041318655014038
o valor final do loss no teste da rede optimizada é  0.7994734048843384
```



Se se entrenaron os 3 modelos verás bastantes outputs e aínda que as tres sexan máis ou menos boas empregaremos só a mellor para dar resultados, ou sexa, consideramos só `modelo_optimizado`.

```
y2_neuron_train_class = np.argmax(modelo3.predict(x2_train_scaled), axis=1)
y2_neuron_train_labels = le.inverse_transform(y2_neuron_train_class)
y2_neuron_test_class = np.argmax(modelo3.predict(x2_test_scaled), axis=1)
y2_neuron_test_labels = le.inverse_transform(y2_neuron_test_class)

print(f'A exactitude de predición da rede neuronal no conxunto de adestramento é
{accuracy_score(y2_train, y2_neuron_train_labels):.4f}')
print(f'A exactitude de predición da rede neuronal no conxunto de teste é
{accuracy_score(y2_test, y2_neuron_test_labels):.4f}\n')

print(
    classification_report(
        y2_test,
        y2_neuron_test_labels,
        target_names=labels_1
    )
)
```

```
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
50/50 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
A exactitude de predición da rede neuronal no conxunto de adestramento é 0.8094
A exactitude de predición da rede neuronal no conxunto de teste é 0.7956

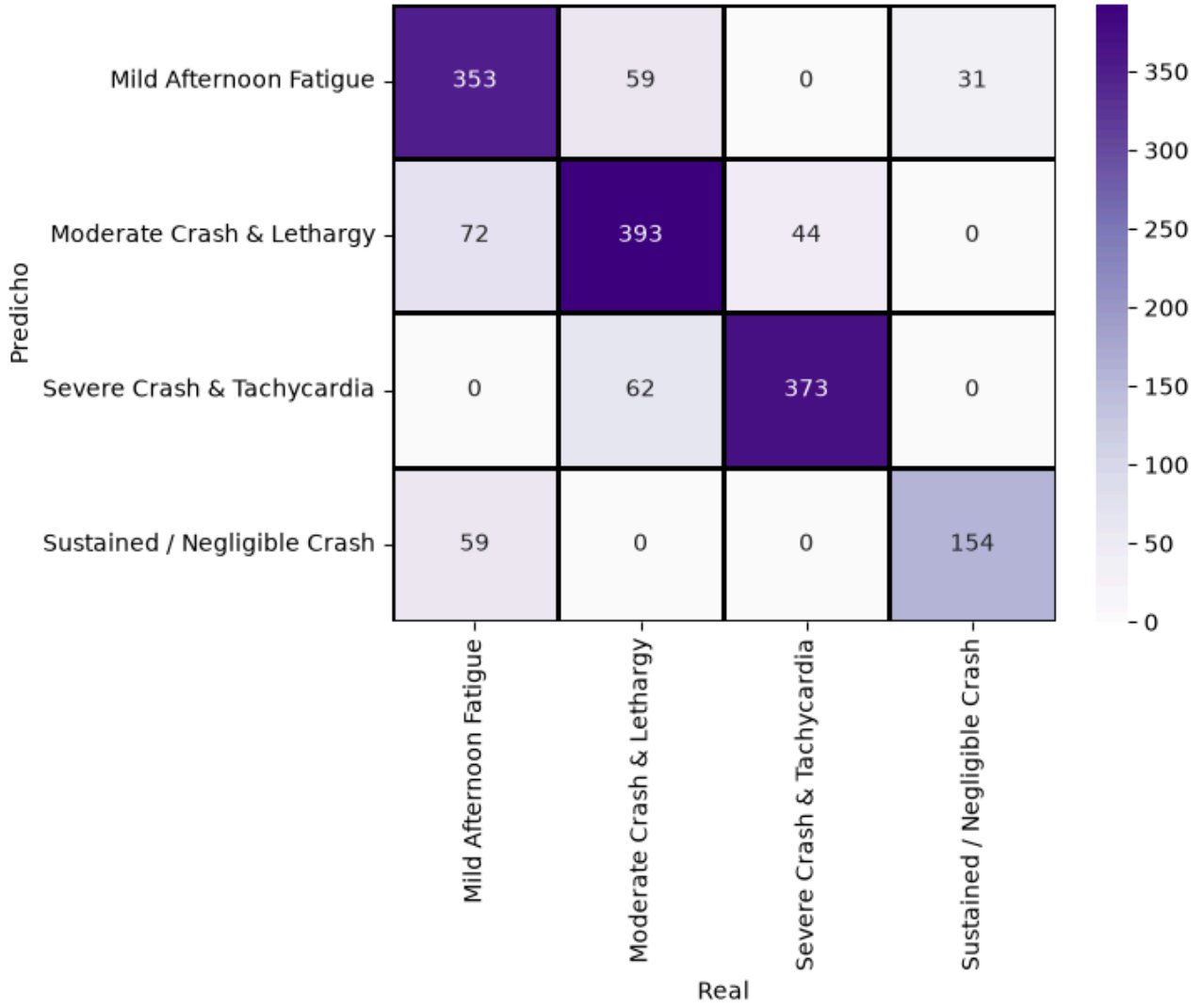
                           precision    recall  f1-score   support

      Mild Afternoon Fatigue       0.73      0.80      0.76       443
   Moderate Crash & Lethargy       0.76      0.77      0.77       509
  Severe Crash & Tachycardia       0.89      0.86      0.88       435
Sustained / Negligible Crash       0.83      0.72      0.77       213

                    accuracy                           0.80      1600
                   macro avg       0.81      0.79      0.79      1600
                weighted avg       0.80      0.80      0.80      1600
```

```
cm_rede = confusion_matrix(y2_test, y2_neuron_test_labels, labels=labels_1)

sns.heatmap(cm_rede, cmap='Purples', annot=True, fmt='d', linewidths=1,
linecolor='black', xticklabels=labels_1, yticklabels=labels_1)
plt.ylabel('Predicho')
plt.xlabel('Real')
plt.show()
```



1.4.7 4.4 Comparación de resultados

Na nosa aplicación das redes neuronais con `keras` para a predición da variable "crash_risk_level" fixemos moito traballo computacional pretendendo refinar e mellorar os nosos modelos. Entre todas as probas, como xa se comentou non atopamos grandes diferenzas salvo a velocidade de compilado. En xeral as redes saturan a súa capacidade de predición sobre os 80 puntos, algo estraño, pero son estables. Podemos visualizar o comportamento do loss e da predición nos modelos no punto 4.3., nótase unha lixeira mellora na rede_optimzada pero nada salientable. De todos xeitos, unha capacidade de predición de 80 puntos é bastante boa o que supón unha correcta aplicación do método.

1.5 5. Conclusións finais

En recapitulación, comparamos as distribucións de datos e logo aplicamos métodos de machine learning para predicir as columnas "crash_risk_level" e "crash_severity_score". Ao aplicar por primeira vez modelos ML de scikit-learn atopamos que predecía correctamente case o 100% das veces o primer tipo de datos, polo que asumimos que era a segunda columna a que lle chivava información ao modelo, tras isto aplicamos as nosas técnicas de novo optimizando os parámetros mediante validación cruzada obtendo resultados moi favorables (sobre os 80 puntos). Entre os 4 modelos que comparamos (linear, rbf, RandomForest e HistGradientBoosting) o primeiro tiña pouca capacidade predictiva e os dous últimos sofrían overfitting, mentras que o modelo rbf era óptimo: rápido, boa predición, estable e sen problemas de entramento. Seguido disto, analizamos con regresións SVR de sklearn a variable descartada "crash_severity_score", da que tamén obtivemos resultados moi favorables; outra vez o modelo linear quedábase curto e o rbf era o óptimo, neste caso en vez de empregar RandomForest e HistGradientBoosting empregamos unha regresión polinómica que axustamos os seus parámetros (ente eles o grao do polinomio) mediante validación cruzada o que supoxo un custe computacional elevadísimo, por tanto, non é un modelo práctico. Por último empregamos as redes neuronais de keras, traballamos ao principio un pouco a cegas intentando atopar a mellor estrutura posible, polo que aplicamos tamén validación cruzada vía keras_tuner, no entanto non obtivemos resultados moi esperanzadores: os mellores modelos contiñan moito ruído e de entre todas as posibilidadess a penas probamos unhas 200 combinacións constándonos máis de 7h de compilación. Finalmente entre proba e erro concluímos 3 modelos, un simple, unha mellora e un optimizado, que non teñen grandes diferenzas entre eles salvo que o modelo optimizado, contra ser unha rede moito máis complexa cas outras, esixe menos tempo de compilación.

1.6 Apéndice Cross Validation en redes neuronais

CAPTION Non correr estas celas é un total de ~7h. Déixoas comentadas por se acaso.

```
# import keras_tuner as kt

# def build_model(hp):

#     l2 = hp.Choice("l2", [0.0, 0.05, 0.01, 0.005, 0.001])

#     model = Sequential([
#         layers.Input(shape=(x2_train_scaled.shape[1],)),

#         layers.Dense(
#             hp.Int("n1", 16, 128, step=16),
```

```
#             activation='relu',
#             kernel_initializer='he_normal',
#             kernel_regularizer=L2(l2)
#         ),

#         layers.Dropout(hp.Choice("dropout", [0.0, 0.1, 0.2])),

#         layers.Dense(
#             hp.Int("n2", 8, 64, step=8),
#             activation='relu', kernel_initializer='he_normal', kernel_regularizer =
L2(l2)
#         ),

#         layers.Dense(16, activation='relu',
#                 kernel_initializer='he_normal',
#                 kernel_regularizer= L2(l2)
#             ),

#         layers.Dense(4, activation='softmax')
#     ])

#     model.compile(
#         optimizer=SGD(
#             learning_rate=hp.Choice("lr", [0.05, 0.01, 0.005, 0.001, 0.0005,
0.0001, 0.00001])
#         ),
#         loss='sparse_categorical_crossentropy',
#         metrics=['accuracy']
#     )

#     return model

# os.makedirs("axuste_keras", exist_ok=True)

# tuner = kt.RandomSearch(
#     build_model,
#     objective='val_accuracy',
#     max_trials=50,
#     executions_per_trial=3,
#     directory="/axuste_keras",
#     project_name="modelo1",
#     overwrite=True
# )

# tuner.search(
#     x2_train_scaled,
#     y2_train_enc,
#     validation_data=(x2_test_scaled, y2_test_enc),
#     epochs=60,
#     batch_size=10
# )
```

```
Search: Running Trial #1

Value             |Best Value So Far |Hyperparameter
0.005             |0.005             |l2
128               |128               |n1
0.1               |0.1               |dropout
8                 |8                 |n2
0.0005            |0.0005            |lr

Epoch 1/60
640/640 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.3033 - loss: 3.1285 -
val_accuracy: 0.3425 - val_loss: 2.9316
Epoch 2/60
640/640 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.3411 - loss: 2.9248 -
val_accuracy: 0.3650 - val_loss: 2.8701
Epoch 3/60
640/640 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.3652 - loss: 2.8714 -
val_accuracy: 0.4081 - val_loss: 2.8265
Epoch 4/60
 71/640 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.4014 - loss: 2.8471
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
Cell In[498], line 54
     50     project_name="modelo1",
     51     overwrite=True
     52 )
     53
---> 54 tuner.search(
     55     x2_train_scaled,
     56     y2_train_enc,
     57     validation_data=(x2_test_scaled, y2_test_enc),

File ~\anaconda3\envs\tensorflow_env\Lib\site-
packages\keras_tuner\src\engine\base_tuner.py:234, in BaseTuner.search(self,
*fit_args, **fit_kwargs)
    231         continue
    233     self.on_trial_begin(trial)
--> 234     self._try_run_and_update_trial(trial, *fit_args, **fit_kwargs)
    235     self.on_trial_end(trial)
    236 self.on_search_end()

File ~\anaconda3\envs\tensorflow_env\Lib\site-
packages\keras_tuner\src\engine\base_tuner.py:274, in
BaseTuner._try_run_and_update_trial(self, trial, *fit_args, **fit_kwargs)
    272 def _try_run_and_update_trial(self, trial, *fit_args, **fit_kwargs):
    273     try:
--> 274         self._run_and_update_trial(trial, *fit_args, **fit_kwargs)
    275         trial.status = trial_module.TrialStatus.COMPLETED
    276         return

File ~\anaconda3\envs\tensorflow_env\Lib\site-
packages\keras_tuner\src\engine\base_tuner.py:239, in
BaseTuner._run_and_update_trial(self, trial, *fit_args, **fit_kwargs)
    238 def _run_and_update_trial(self, trial, *fit_args, **fit_kwargs):
```

```
──> 239     results = self.run_trial(trial, *fit_args, **fit_kwargs)
    240     if self.oracle.get_trial(trial.trial_id).metrics.exists(
    241         self.oracle.objective.name
    242     ):
    243         # The oracle is updated by calling `self.oracle.update_trial()` in
    244         # `Tuner.run_trial()`. For backward compatibility, we support this
    245         # use case. No further action needed in this case.
    246         warnings.warn(
    247             "The use case of calling "
    248             "`self.oracle.update_trial(trial_id, metrics)` "
   (...)    254             stacklevel=2,
    255         )

File ~\anaconda3\envs\tensorflow_env\Lib\site-
packages\keras_tuner\src\engine\tuner.py:314, in Tuner.run_trial(self, trial, *args,
**kwargs)
    312     callbacks.append(model_checkpoint)
    313     copied_kwargs["callbacks"] = callbacks
──> 314     obj_value = self._build_and_fit_model(trial, *args, **copied_kwargs)
    316     histories.append(obj_value)
    317 return histories

File ~\anaconda3\envs\tensorflow_env\Lib\site-
packages\keras_tuner\src\engine\tuner.py:233, in Tuner._build_and_fit_model(self,
trial, *args, **kwargs)
    231 hp = trial.hyperparameters
    232 model = self._try_build(hp)
──> 233 results = self.hypermodel.fit(hp, model, *args, **kwargs)
    235 # Save the build config for model loading later.
    236 if backend.config.multi_backend():

File ~\anaconda3\envs\tensorflow_env\Lib\site-
packages\keras_tuner\src\engine\hypermodel.py:149, in HyperModel.fit(self, hp, model,
*args, **kwargs)
    125 def fit(self, hp, model, *args, **kwargs):
    126     """Train the model.
    127
    128     Args:
   (...)    147         If return a float, it should be the `objective` value.
    148     """
──> 149     return model.fit(*args, **kwargs)

File ~\anaconda3\envs\tensorflow_env\Lib\site-
packages\keras\src\utils\traceback_utils.py:117, in filter_traceback.
<locals>.error_handler(*args, **kwargs)
    115 filtered_tb = None
    116 try:
──> 117     return fn(*args, **kwargs)
    118 except Exception as e:
    119     filtered_tb = _process_traceback_frames(e.__traceback__)

File ~\anaconda3\envs\tensorflow_env\Lib\site-
packages\keras\src\backend\tensorflow\trainer.py:399, in TensorFlowTrainer.fit(self,
x, y, batch_size, epochs, verbose, callbacks, validation_split, validation_data,
shuffle, class_weight, sample_weight, initial_epoch, steps_per_epoch,
```

```
validation_steps, validation_batch_size, validation_freq)
    397 for begin_step, end_step, iterator in epoch_iterator:
    398     callbacks.on_train_batch_begin(begin_step)
--> 399     logs = self.train_function(iterator)
    400     callbacks.on_train_batch_end(end_step, logs)
    401     if self.stop_training:

File ~\anaconda3\envs\tensorflow_env\Lib\site-
packages\keras\src\backend\tensorflow\trainer.py:244, in
TensorFlowTrainer._make_function.<locals>.function(iterator)
    242     if not opt_outputs.has_value():
    243         raise StopIteration
--> 244     return opt_outputs.get_value()
    245 else:
    246     for step, data in zip(
    247         range(self.steps_per_execution), iterator
    248     ):

File ~\anaconda3\envs\tensorflow_env\Lib\site-
packages\tensorflow\python\data\ops\optional_ops.py:186, in
_OptionalImpl.get_value(self, name)
    183 with ops.name_scope(name, "OptionalGetValue",
    184                     [self._variant_tensor]) as scope:
    185   with ops.colocate_with(self._variant_tensor):
--> 186     result = gen_optional_ops.optional_get_value(
    187         self._variant_tensor,
    188         name=scope,
    189         output_types=structure.get_flat_tensor_types(self._element_spec),
    190         output_shapes=structure.get_flat_tensor_shapes(self._element_spec),
    191     )
    192   # NOTE: We do not colocate the deserialization of composite tensors
    193   # because not all ops are guaranteed to have non-GPU kernels.
    194   return structure.from_tensor_list(self._element_spec, result)

File ~\anaconda3\envs\tensorflow_env\Lib\site-
packages\tensorflow\python\ops\gen_optional_ops.py:95, in optional_get_value(optional,
output_types, output_shapes, name)
     93 if tld.is_eager:
     94   try:
---> 95     _result = pywrap_tfe.TFE_Py_FastPathExecute(
     96       _ctx, "OptionalGetValue", name, optional, "output_types",
     97       output_types, "output_shapes", output_shapes)
     98     return _result
     99   except _core._NotOkStatusException as e:

KeyboardInterrupt:
```

```
# best_model = tuner.get_best_models(1)[0]
# cerebro = best_model.fit(
#     x2_train_scaled,
#     y2_train_enc,
#     validation_data=(x2_test_scaled, y2_test_enc),
#     epochs=80,
#     batch_size=8
# )

# plt.plot(cerebro.history['loss'])
# plt.plot(cerebro.history['val_loss'])

# print(cerebro.history['accuracy'][-1])
# print(cerebro.history['val_accuracy'][-1])
# print(cerebro.history['loss'][-1])
# print(cerebro.history['val_loss'][-1])

# plt.figure()

# plt.plot(cerebro.history['accuracy'])
# plt.plot(cerebro.history['val_accuracy'])
```

```
Epoch 1/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8037 - loss: 0.6388 -
val_accuracy: 0.7931 - val_loss: 0.6459
Epoch 2/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7983 - loss: 0.6223 -
val_accuracy: 0.7956 - val_loss: 0.6270
Epoch 3/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8016 - loss: 0.6067 -
val_accuracy: 0.7919 - val_loss: 0.6174
Epoch 4/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8022 - loss: 0.5923 -
val_accuracy: 0.7869 - val_loss: 0.6042
Epoch 5/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8037 - loss: 0.5842 -
val_accuracy: 0.7956 - val_loss: 0.5909
Epoch 6/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8012 - loss: 0.5748 -
val_accuracy: 0.7937 - val_loss: 0.5803
Epoch 7/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8031 - loss: 0.5633 -
val_accuracy: 0.7919 - val_loss: 0.5782
Epoch 8/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8067 - loss: 0.5563 -
val_accuracy: 0.8031 - val_loss: 0.5675
Epoch 9/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8009 - loss: 0.5548 -
val_accuracy: 0.8044 - val_loss: 0.5693
Epoch 10/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8056 - loss: 0.5431 -
val_accuracy: 0.7969 - val_loss: 0.5538
Epoch 11/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8095 - loss: 0.5392 -
```

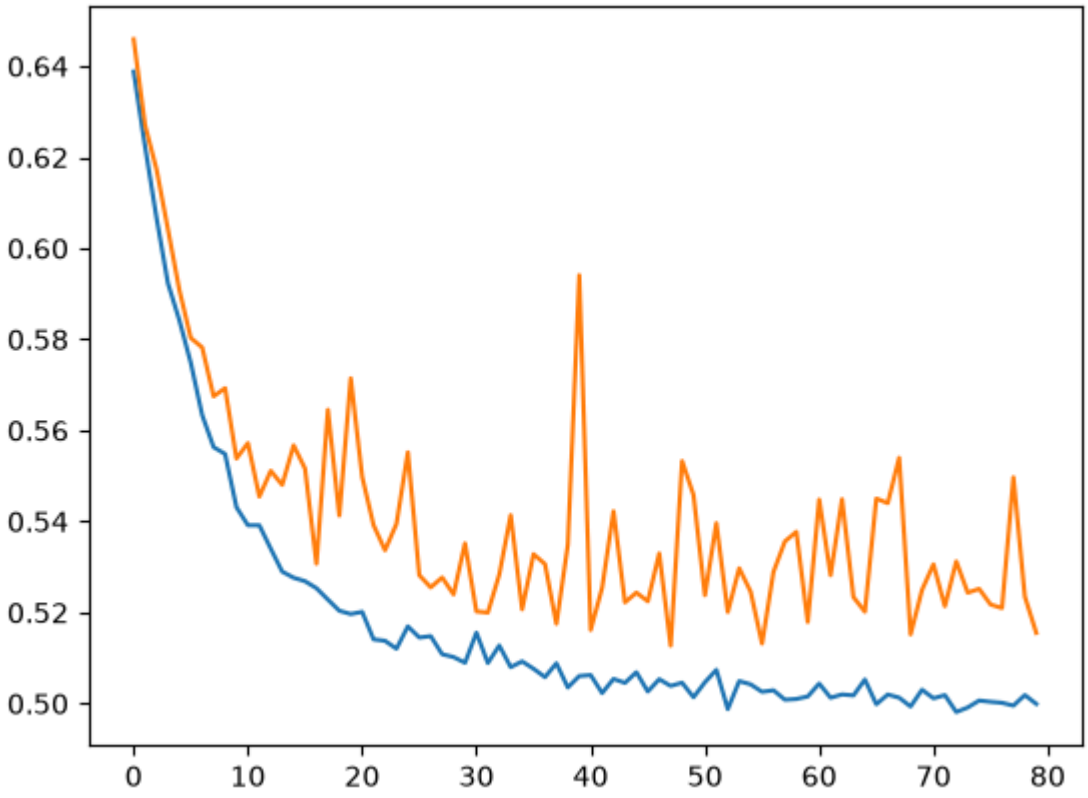
```
val_accuracy: 0.7937 - val_loss: 0.5572
Epoch 12/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8067 - loss: 0.5391 -
val_accuracy: 0.8006 - val_loss: 0.5454
Epoch 13/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8059 - loss: 0.5340 -
val_accuracy: 0.7987 - val_loss: 0.5511
Epoch 14/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8023 - loss: 0.5290 -
val_accuracy: 0.7931 - val_loss: 0.5480
Epoch 15/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8047 - loss: 0.5276 -
val_accuracy: 0.7919 - val_loss: 0.5567
Epoch 16/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8008 - loss: 0.5269 -
val_accuracy: 0.7900 - val_loss: 0.5516
Epoch 17/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8044 - loss: 0.5253 -
val_accuracy: 0.8056 - val_loss: 0.5307
Epoch 18/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8044 - loss: 0.5228 -
val_accuracy: 0.7775 - val_loss: 0.5645
Epoch 19/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8062 - loss: 0.5204 -
val_accuracy: 0.7906 - val_loss: 0.5413
Epoch 20/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8056 - loss: 0.5197 -
val_accuracy: 0.7837 - val_loss: 0.5714
Epoch 21/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8045 - loss: 0.5201 -
val_accuracy: 0.7831 - val_loss: 0.5498
Epoch 22/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8059 - loss: 0.5141 -
val_accuracy: 0.7912 - val_loss: 0.5392
Epoch 23/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8041 - loss: 0.5138 -
val_accuracy: 0.7881 - val_loss: 0.5337
Epoch 24/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8064 - loss: 0.5120 -
val_accuracy: 0.7912 - val_loss: 0.5395
Epoch 25/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8059 - loss: 0.5169 -
val_accuracy: 0.7850 - val_loss: 0.5552
Epoch 26/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8089 - loss: 0.5145 -
val_accuracy: 0.7912 - val_loss: 0.5282
Epoch 27/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8058 - loss: 0.5148 -
val_accuracy: 0.7969 - val_loss: 0.5255
Epoch 28/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8031 - loss: 0.5108 -
val_accuracy: 0.7969 - val_loss: 0.5277
Epoch 29/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8042 - loss: 0.5102 -
val_accuracy: 0.7912 - val_loss: 0.5240
```

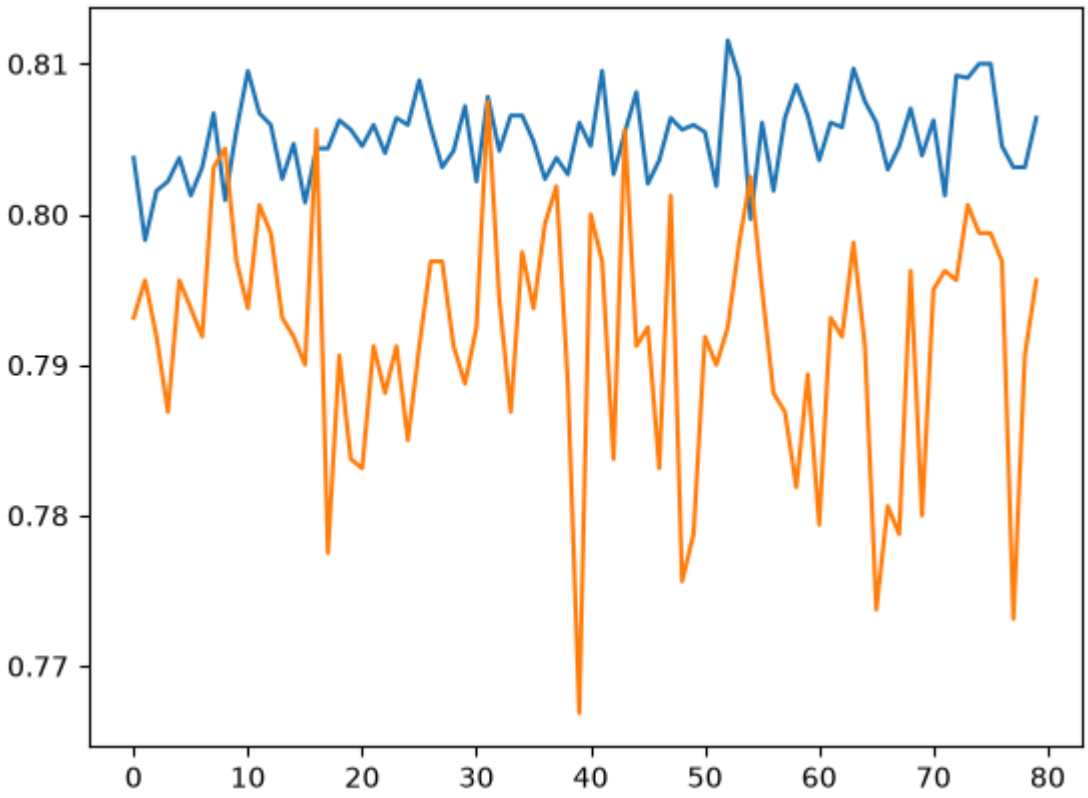
```
Epoch 30/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8072 - loss: 0.5089 -
val_accuracy: 0.7887 - val_loss: 0.5352
Epoch 31/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8022 - loss: 0.5156 -
val_accuracy: 0.7925 - val_loss: 0.5202
Epoch 32/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8078 - loss: 0.5089 -
val_accuracy: 0.8075 - val_loss: 0.5200
Epoch 33/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8042 - loss: 0.5127 -
val_accuracy: 0.7944 - val_loss: 0.5283
Epoch 34/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8066 - loss: 0.5080 -
val_accuracy: 0.7869 - val_loss: 0.5414
Epoch 35/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8066 - loss: 0.5092 -
val_accuracy: 0.7975 - val_loss: 0.5207
Epoch 36/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8048 - loss: 0.5076 -
val_accuracy: 0.7937 - val_loss: 0.5328
Epoch 37/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8023 - loss: 0.5058 -
val_accuracy: 0.7994 - val_loss: 0.5306
Epoch 38/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8037 - loss: 0.5088 -
val_accuracy: 0.8019 - val_loss: 0.5175
Epoch 39/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8027 - loss: 0.5035 -
val_accuracy: 0.7887 - val_loss: 0.5348
Epoch 40/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8061 - loss: 0.5060 -
val_accuracy: 0.7669 - val_loss: 0.5941
Epoch 41/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8045 - loss: 0.5063 -
val_accuracy: 0.8000 - val_loss: 0.5162
Epoch 42/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8095 - loss: 0.5023 -
val_accuracy: 0.7969 - val_loss: 0.5254
Epoch 43/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8027 - loss: 0.5054 -
val_accuracy: 0.7837 - val_loss: 0.5422
Epoch 44/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8055 - loss: 0.5045 -
val_accuracy: 0.8056 - val_loss: 0.5222
Epoch 45/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8081 - loss: 0.5069 -
val_accuracy: 0.7912 - val_loss: 0.5244
Epoch 46/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8020 - loss: 0.5026 -
val_accuracy: 0.7925 - val_loss: 0.5225
Epoch 47/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8036 - loss: 0.5053 -
val_accuracy: 0.7831 - val_loss: 0.5330
Epoch 48/80
```

```
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8064 - loss: 0.5039 -
val_accuracy: 0.8012 - val_loss: 0.5127
Epoch 49/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8056 - loss: 0.5046 -
val_accuracy: 0.7756 - val_loss: 0.5533
Epoch 50/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8059 - loss: 0.5014 -
val_accuracy: 0.7788 - val_loss: 0.5458
Epoch 51/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8055 - loss: 0.5047 -
val_accuracy: 0.7919 - val_loss: 0.5238
Epoch 52/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8019 - loss: 0.5074 -
val_accuracy: 0.7900 - val_loss: 0.5396
Epoch 53/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8116 - loss: 0.4988 -
val_accuracy: 0.7925 - val_loss: 0.5201
Epoch 54/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8091 - loss: 0.5049 -
val_accuracy: 0.7981 - val_loss: 0.5297
Epoch 55/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7997 - loss: 0.5042 -
val_accuracy: 0.8025 - val_loss: 0.5246
Epoch 56/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8061 - loss: 0.5026 -
val_accuracy: 0.7950 - val_loss: 0.5132
Epoch 57/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8016 - loss: 0.5029 -
val_accuracy: 0.7881 - val_loss: 0.5289
Epoch 58/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8064 - loss: 0.5008 -
val_accuracy: 0.7869 - val_loss: 0.5356
Epoch 59/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8086 - loss: 0.5010 -
val_accuracy: 0.7819 - val_loss: 0.5377
Epoch 60/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8066 - loss: 0.5015 -
val_accuracy: 0.7894 - val_loss: 0.5179
Epoch 61/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8036 - loss: 0.5044 -
val_accuracy: 0.7794 - val_loss: 0.5448
Epoch 62/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8061 - loss: 0.5013 -
val_accuracy: 0.7931 - val_loss: 0.5282
Epoch 63/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8058 - loss: 0.5020 -
val_accuracy: 0.7919 - val_loss: 0.5449
Epoch 64/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8097 - loss: 0.5018 -
val_accuracy: 0.7981 - val_loss: 0.5234
Epoch 65/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8075 - loss: 0.5053 -
val_accuracy: 0.7912 - val_loss: 0.5202
Epoch 66/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8061 - loss: 0.4998 -
```

```
val_accuracy: 0.7738 - val_loss: 0.5450
Epoch 67/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8030 - loss: 0.5020 -
val_accuracy: 0.7806 - val_loss: 0.5440
Epoch 68/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8045 - loss: 0.5013 -
val_accuracy: 0.7788 - val_loss: 0.5540
Epoch 69/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8070 - loss: 0.4993 -
val_accuracy: 0.7962 - val_loss: 0.5152
Epoch 70/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8039 - loss: 0.5030 -
val_accuracy: 0.7800 - val_loss: 0.5249
Epoch 71/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8062 - loss: 0.5011 -
val_accuracy: 0.7950 - val_loss: 0.5306
Epoch 72/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8012 - loss: 0.5018 -
val_accuracy: 0.7962 - val_loss: 0.5214
Epoch 73/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8092 - loss: 0.4981 -
val_accuracy: 0.7956 - val_loss: 0.5312
Epoch 74/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8091 - loss: 0.4991 -
val_accuracy: 0.8006 - val_loss: 0.5243
Epoch 75/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8100 - loss: 0.5007 -
val_accuracy: 0.7987 - val_loss: 0.5252
Epoch 76/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8100 - loss: 0.5004 -
val_accuracy: 0.7987 - val_loss: 0.5217
Epoch 77/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8045 - loss: 0.5002 -
val_accuracy: 0.7969 - val_loss: 0.5210
Epoch 78/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8031 - loss: 0.4995 -
val_accuracy: 0.7731 - val_loss: 0.5497
Epoch 79/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8031 - loss: 0.5019 -
val_accuracy: 0.7906 - val_loss: 0.5235
Epoch 80/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8064 - loss: 0.4999 -
val_accuracy: 0.7956 - val_loss: 0.5155
0.8064062595367432
0.7956249713897705
0.4999177157878876
0.515512228012085

[<matplotlib.lines.Line2D at 0x1fa68824a50>]
```






```
# def build_model_modified(hp):
#     lr1 = hp.Choice("l1", [0.0, 0.01, 0.001, 0.0001])
#     lr2 = hp.Choice("l2", [0.0, 0.05, 0.01, 0.005, 0.001])

#     model = Sequential([
#         layers.Input(shape=(x2_train_scaled.shape[1],)),
```

```
#         layers.Dense(hp.Int("n1", 32, 256, step=32),
#                      activation='softplus', kernel_initializer='glorot_uniform' ,
#                      kernel_regularizer= L1L2(l1 = lr1, l2 = lr2)),
#         layers.Dense(hp.Int("n2", 8, 64, step=8),
#                      activation='elu', kernel_initializer='he_normal',
#                      kernel_regularizer= L2(lr2)),
#         layers.Dense(hp.Int("n3", 16, 64, step=8),
#                      activation='leaky_relu', kernel_initializer='he_normal',
#                      kernel_regularizer= L2(lr2)),
#         layers.Dense(hp.Int("n4", 12, 32, step=4),
#                      activation='leaky_relu', kernel_initializer='he_normal',
#                      kernel_regularizer= L2(lr2)),

#         layers.Dense(4, activation='softmax')
#     ])

#     model.compile(
#         optimizer=Adam(
#             learning_rate=hp.Choice("lr", [0.05, 0.01, 0.005, 0.001, 0.0005,
0.0001])
#         ),
#         loss='sparse_categorical_crossentropy',
#         metrics=['accuracy']
#     )

#     return model

# os.makedirs("axuste_keras", exist_ok=True)

# tuner = kt.RandomSearch(
#     build_model_modified,
#     objective='val_accuracy',
#     max_trials=50,
#     executions_per_trial=3,
#     directory="/axuste_keras",
#     project_name="modelo2",
#     overwrite=True
# )

# tuner.search(
#     x2_train_scaled,
#     y2_train_enc,
#     validation_data=(x2_test_scaled, y2_test_enc),
#     epochs=60,
#     batch_size=8
# )
```

```
Trial 50 Complete [00h 05m 24s]
val_accuracy: 0.7981250087420145

Best val_accuracy So Far: 0.8079166611035665
Total elapsed time: 04h 26m 16s
```

```
# best_model = tuner.get_best_models(1)[0]
# cerebro = best_model.fit(
#     x2_train_scaled,
#     y2_train_enc,
#     validation_data=(x2_test_scaled, y2_test_enc),
#     epochs=80,
#     batch_size=8
# )

# plt.plot(cerebro.history['loss'])
# plt.plot(cerebro.history['val_loss'])

# print(cerebro.history['accuracy'][-1])
# print(cerebro.history['val_accuracy'][-1])
# print(cerebro.history['loss'][-1])
# print(cerebro.history['val_loss'][-1])

# plt.figure()

# plt.plot(cerebro.history['accuracy'])
# plt.plot(cerebro.history['val_accuracy'])
```

```
Epoch 1/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.7998 - loss: 0.4921 -
val_accuracy: 0.7856 - val_loss: 0.5011
Epoch 2/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7994 - loss: 0.4835 -
val_accuracy: 0.7875 - val_loss: 0.5199
Epoch 3/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7975 - loss: 0.4887 -
val_accuracy: 0.7944 - val_loss: 0.4926
Epoch 4/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7994 - loss: 0.4866 -
val_accuracy: 0.7956 - val_loss: 0.5030
Epoch 5/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7972 - loss: 0.4862 -
val_accuracy: 0.7894 - val_loss: 0.5057
Epoch 6/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7933 - loss: 0.4887 -
val_accuracy: 0.7706 - val_loss: 0.5375
Epoch 7/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7991 - loss: 0.4854 -
val_accuracy: 0.7894 - val_loss: 0.5047
Epoch 8/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7972 - loss: 0.4879 -
val_accuracy: 0.7869 - val_loss: 0.5166
Epoch 9/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8044 - loss: 0.4810 -
val_accuracy: 0.7900 - val_loss: 0.5073
Epoch 10/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8011 - loss: 0.4882 -
val_accuracy: 0.7887 - val_loss: 0.4912
Epoch 11/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7994 - loss: 0.4841 -
```

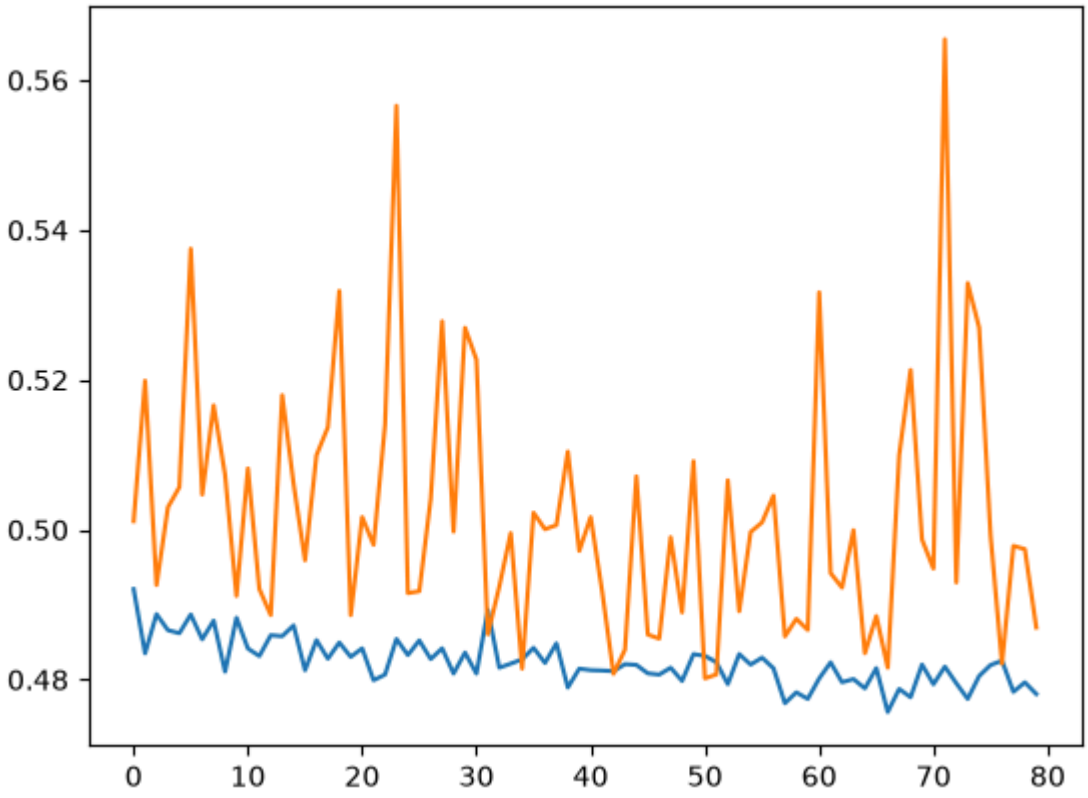
```
val_accuracy: 0.7844 - val_loss: 0.5082
Epoch 12/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8027 - loss: 0.4831 -
val_accuracy: 0.7956 - val_loss: 0.4921
Epoch 13/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7994 - loss: 0.4859 -
val_accuracy: 0.8019 - val_loss: 0.4886
Epoch 14/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7998 - loss: 0.4858 -
val_accuracy: 0.7881 - val_loss: 0.5180
Epoch 15/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8003 - loss: 0.4872 -
val_accuracy: 0.7887 - val_loss: 0.5061
Epoch 16/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7997 - loss: 0.4812 -
val_accuracy: 0.7969 - val_loss: 0.4959
Epoch 17/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8003 - loss: 0.4853 -
val_accuracy: 0.7894 - val_loss: 0.5099
Epoch 18/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7998 - loss: 0.4828 -
val_accuracy: 0.7850 - val_loss: 0.5137
Epoch 19/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7978 - loss: 0.4849 -
val_accuracy: 0.7837 - val_loss: 0.5319
Epoch 20/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8025 - loss: 0.4830 -
val_accuracy: 0.8006 - val_loss: 0.4886
Epoch 21/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7973 - loss: 0.4842 -
val_accuracy: 0.7912 - val_loss: 0.5017
Epoch 22/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7984 - loss: 0.4799 -
val_accuracy: 0.8025 - val_loss: 0.4980
Epoch 23/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8044 - loss: 0.4807 -
val_accuracy: 0.7700 - val_loss: 0.5140
Epoch 24/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7948 - loss: 0.4854 -
val_accuracy: 0.7644 - val_loss: 0.5566
Epoch 25/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8008 - loss: 0.4833 -
val_accuracy: 0.8025 - val_loss: 0.4915
Epoch 26/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7991 - loss: 0.4852 -
val_accuracy: 0.8006 - val_loss: 0.4918
Epoch 27/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7987 - loss: 0.4827 -
val_accuracy: 0.7919 - val_loss: 0.5041
Epoch 28/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8000 - loss: 0.4842 -
val_accuracy: 0.7750 - val_loss: 0.5278
Epoch 29/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7980 - loss: 0.4808 -
val_accuracy: 0.7856 - val_loss: 0.4997
```

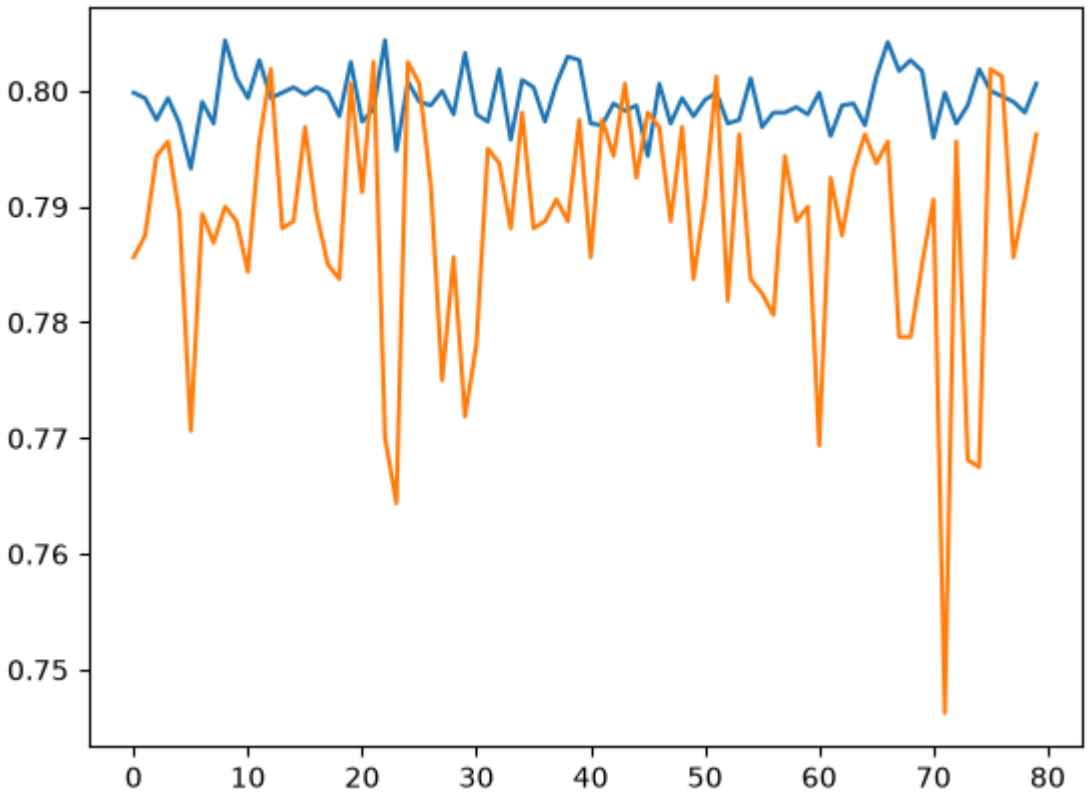
```
Epoch 30/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8033 - loss: 0.4836 -
val_accuracy: 0.7719 - val_loss: 0.5270
Epoch 31/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7980 - loss: 0.4808 -
val_accuracy: 0.7781 - val_loss: 0.5227
Epoch 32/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7973 - loss: 0.4893 -
val_accuracy: 0.7950 - val_loss: 0.4860
Epoch 33/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8019 - loss: 0.4816 -
val_accuracy: 0.7937 - val_loss: 0.4924
Epoch 34/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7958 - loss: 0.4821 -
val_accuracy: 0.7881 - val_loss: 0.4995
Epoch 35/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8009 - loss: 0.4827 -
val_accuracy: 0.7981 - val_loss: 0.4815
Epoch 36/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8003 - loss: 0.4842 -
val_accuracy: 0.7881 - val_loss: 0.5023
Epoch 37/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7973 - loss: 0.4822 -
val_accuracy: 0.7887 - val_loss: 0.5001
Epoch 38/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8006 - loss: 0.4848 -
val_accuracy: 0.7906 - val_loss: 0.5006
Epoch 39/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8030 - loss: 0.4789 -
val_accuracy: 0.7887 - val_loss: 0.5104
Epoch 40/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8027 - loss: 0.4814 -
val_accuracy: 0.7975 - val_loss: 0.4972
Epoch 41/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7972 - loss: 0.4813 -
val_accuracy: 0.7856 - val_loss: 0.5017
Epoch 42/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7970 - loss: 0.4812 -
val_accuracy: 0.7975 - val_loss: 0.4918
Epoch 43/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7989 - loss: 0.4811 -
val_accuracy: 0.7944 - val_loss: 0.4808
Epoch 44/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7983 - loss: 0.4821 -
val_accuracy: 0.8006 - val_loss: 0.4840
Epoch 45/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7987 - loss: 0.4820 -
val_accuracy: 0.7925 - val_loss: 0.5071
Epoch 46/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7944 - loss: 0.4808 -
val_accuracy: 0.7981 - val_loss: 0.4860
Epoch 47/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8006 - loss: 0.4807 -
val_accuracy: 0.7969 - val_loss: 0.4855
Epoch 48/80
```

```
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7972 - loss: 0.4816 -
val_accuracy: 0.7887 - val_loss: 0.4990
Epoch 49/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7994 - loss: 0.4798 -
val_accuracy: 0.7969 - val_loss: 0.4890
Epoch 50/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7978 - loss: 0.4834 -
val_accuracy: 0.7837 - val_loss: 0.5091
Epoch 51/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7992 - loss: 0.4832 -
val_accuracy: 0.7906 - val_loss: 0.4802
Epoch 52/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7998 - loss: 0.4823 -
val_accuracy: 0.8012 - val_loss: 0.4807
Epoch 53/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7972 - loss: 0.4794 -
val_accuracy: 0.7819 - val_loss: 0.5066
Epoch 54/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7975 - loss: 0.4834 -
val_accuracy: 0.7962 - val_loss: 0.4891
Epoch 55/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8011 - loss: 0.4820 -
val_accuracy: 0.7837 - val_loss: 0.4997
Epoch 56/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7969 - loss: 0.4829 -
val_accuracy: 0.7825 - val_loss: 0.5010
Epoch 57/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7981 - loss: 0.4815 -
val_accuracy: 0.7806 - val_loss: 0.5046
Epoch 58/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7981 - loss: 0.4768 -
val_accuracy: 0.7944 - val_loss: 0.4858
Epoch 59/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7986 - loss: 0.4783 -
val_accuracy: 0.7887 - val_loss: 0.4881
Epoch 60/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7980 - loss: 0.4774 -
val_accuracy: 0.7900 - val_loss: 0.4866
Epoch 61/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7998 - loss: 0.4801 -
val_accuracy: 0.7694 - val_loss: 0.5317
Epoch 62/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7961 - loss: 0.4823 -
val_accuracy: 0.7925 - val_loss: 0.4942
Epoch 63/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7987 - loss: 0.4796 -
val_accuracy: 0.7875 - val_loss: 0.4923
Epoch 64/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7989 - loss: 0.4801 -
val_accuracy: 0.7931 - val_loss: 0.4999
Epoch 65/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7970 - loss: 0.4788 -
val_accuracy: 0.7962 - val_loss: 0.4835
Epoch 66/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8012 - loss: 0.4815 -
```

```
val_accuracy: 0.7937 - val_loss: 0.4885
Epoch 67/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8042 - loss: 0.4757 -
val_accuracy: 0.7956 - val_loss: 0.4816
Epoch 68/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8017 - loss: 0.4787 -
val_accuracy: 0.7788 - val_loss: 0.5099
Epoch 69/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8027 - loss: 0.4776 -
val_accuracy: 0.7788 - val_loss: 0.5213
Epoch 70/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8017 - loss: 0.4820 -
val_accuracy: 0.7850 - val_loss: 0.4987
Epoch 71/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7959 - loss: 0.4794 -
val_accuracy: 0.7906 - val_loss: 0.4948
Epoch 72/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7998 - loss: 0.4817 -
val_accuracy: 0.7462 - val_loss: 0.5655
Epoch 73/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7972 - loss: 0.4795 -
val_accuracy: 0.7956 - val_loss: 0.4930
Epoch 74/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7987 - loss: 0.4774 -
val_accuracy: 0.7681 - val_loss: 0.5329
Epoch 75/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8019 - loss: 0.4805 -
val_accuracy: 0.7675 - val_loss: 0.5270
Epoch 76/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8000 - loss: 0.4819 -
val_accuracy: 0.8019 - val_loss: 0.4992
Epoch 77/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7995 - loss: 0.4825 -
val_accuracy: 0.8012 - val_loss: 0.4821
Epoch 78/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7991 - loss: 0.4784 -
val_accuracy: 0.7856 - val_loss: 0.4978
Epoch 79/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7981 - loss: 0.4796 -
val_accuracy: 0.7906 - val_loss: 0.4974
Epoch 80/80
800/800 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8006 - loss: 0.4781 -
val_accuracy: 0.7962 - val_loss: 0.4869
0.8006250262260437
0.7962499856948853
0.4780677556991577
0.4869442880153656

[<matplotlib.lines.Line2D at 0x1fad3d0e6d0>]
```


0.56
0.54
0.52
0.50
0.48
0
10
20
30
40
50
60
70
80


0.80
0.79
0.78
0.77
0.76
0.75
0
10
20
30
40
50
60
70
80